

Chapter 9 challenging problem

1. Address of Elements in an Array

Write a program to initialize an array of integers with 10 numbers. The program should then display the number of bytes occupied by the array. Also display each element as well as the address in which it is stored.

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int arr[]={0,1,2,3,4,5,6,7,8,9};
    int num=sizeof(arr);
    cout<<num<<endl;
    for (int i = 0;i<10;i++)
    {
        cout<<arr[i]<<" "<<&arr[i]<<endl;
    }
    return 0;
}
```

2. Test Scores #1

Write a program that dynamically allocates an array large enough to hold a user defined number of test scores. Once all the scores are entered, the array should be passed to a function that sorts them in ascending order. Another function should be called that calculates the average score. The program should display the sorted list of scores and averages with appropriate headings. Use pointer notation rather than array notation whenever possible.

Input Validation: Do not accept negative numbers for test scores.

```
#include <iostream>
#include <iomanip>
using namespace std;
void sort(double*, int);
double average(double*, int);
int main()
{
    double *ScorePtr = nullptr;
    int Scores = 0;
    double Average = 0;
    cout << "How many test scores will you enter? " << endl;
    cin >> Scores;
    while (Scores < 0)
    {
        cout << "The number cannot be negative." << endl;
        cin >> Scores;
    }
    ScorePtr = new double[Scores];
    for (int i = 0; i < Scores; i++)
    {
        cout << "Enter test score " << (i + 1) << ": ";
        cin >> *(ScorePtr+i);
        while (*(ScorePtr+i) < 0)
        {
            cout << "Negative scores are not allowed." << endl;
            cin >> *(ScorePtr+i);
        }
    }
    sort(ScorePtr, Scores);
    Average = average(ScorePtr, Scores);
    cout << "\nThe test scores in ascending " << "order, and their average, are:" << endl;
    for (int j = 0; j < Scores; j++)
    {
        cout << fixed << setprecision(2) << setw(6) <<
            *(ScorePtr+j) << endl;
    }
}
```

```

    }
    cout << "Average score: " << setprecision(2) << fixed <<
        Average << endl << endl;
    delete [] ScorePtr;
    ScorePtr = nullptr;
    return 0;
}
void sort(double* score, int size)
{
    int start, minIndex;
    double minValue;
    // sorting in ascending order
    for (start = 0; start < (size - 1); start++)
    {
        minIndex = start;
        minValue = *(score+start);

        for(int index = start + 1; index < size; index++)
        {
            if (*(score+index) < minValue)
            {
                minValue = *(score+index);
                minIndex = index;
            }
        }

        *(score+minIndex) = *(score+start);
        *(score+start) = minValue;
    }
}
double average(double* score, int numScores)
{
    double total = 0;
    for (int k = 0; k < numScores; k++)
        total += *(score+k);

    // Return the average score.
    return (total / numScores);
}
```

3. Drop Lowest Score

Modify Problem 2 above so the lowest test score is dropped. This score should not be included in the calculation of the average.

```
#include <iostream>
#include <iomanip>
using namespace std;
void sort(double*, int);
double average(double*, int);
int main()
{
    double *ScorePtr = nullptr;
    int Scores = 0;
    double Average = 0;
    cout << "How many test scores will you enter? " << endl;
    cin >> Scores;
    while (Scores < 0)
    {
        cout << "The number cannot be negative." << endl;
        cin >> Scores;
    }
    ScorePtr = new double[Scores];
    for (int i = 0; i < Scores; i++)
    {
        cout << "Enter test score " << (i + 1) << ": ";
        cin >> *(ScorePtr+i);
        while (*(ScorePtr+i) < 0)
        {

```

```

        cout << "Negative scores are not allowed."<<
endl;
        cin >> *(ScorePtr+i);
    }
}
sort(ScorePtr, Scores);
Average = average(ScorePtr, Scores);
cout << "\nThe test scores in ascending "<< "order, and
their average, are:"<< endl;
for (int j = 0; j < Scores; j++)
{
    cout << fixed << setprecision(2) << setw(6) <<
*(ScorePtr+j) << endl;
}
cout << "Average score:  " << setprecision(2) << fixed <<
Average << endl << endl;
delete [] ScorePtr;
ScorePtr = nullptr;
return 0;
}
void sort(double* score, int size)
{
    int start, minIndex;
    double minValue;
    // sorting in ascending order
    for (start = 0; start < (size - 1); start++)
    {
        minIndex = start;
        minValue = *(score+start);

        for(int index = start + 1; index < size; index++)
        {
            if (*(score+index) < minValue)
            {
                minValue = *(score+index);
                minIndex = index;
            }
        }

        *(score+minIndex) = *(score+start);
        *(score+start) = minValue;
    }
}
double average(double* score, int numScores)
{
    double total = 0;
    for (int k = 1; k < numScores; k++)
        total += *(score+k);

    // Return the average score.
    return (total / (numScores-1));
}

```

4. Test Scores #2

Modify the program of Programming Challenge 2 (Test Scores #1) to allow the user to enter name–score pairs. For each student taking a test, the user types the student’s name followed by the student’s integer test score. Modify the sorting function so it takes an array holding the student names, and an array holding the student test scores. When the sorted list of scores is displayed, each student’s name should be displayed along with his or her score. In stepping through the arrays, use pointers rather than array subscripts.

```

#include <iostream>
#include <iomanip>
using namespace std;
struct stu
{

```

```

    int score;
    string name;
};
void sort(stu*, int);
double average(stu*, int);
int main()
{
    stu *ScorePtr = nullptr;
    int Scores = 0;
    double Average = 0;
    cout << "How many test scores will you enter?"<< endl;
    cin >> Scores;
    while (Scores < 0)
    {
        cout << "The number cannot be negative."<< endl;
        cin >> Scores;
    }
    ScorePtr = new stu[Scores];
    for (int i = 0; i < Scores; i++)
    {
        cout << "Enter test score "<< (i + 1) << ": ";
        cin >> (ScorePtr+i)->score;
        cin >> (ScorePtr+i)->name;
        while ((ScorePtr+i)->score < 0)
        {
            cout << "Negative scores are not allowed."<<
endl;
            cin >> (ScorePtr+i)->score;
        }
    }
    sort(ScorePtr, Scores);
    Average = average(ScorePtr, Scores);
    cout << "\nThe test scores in ascending "<< "order, and
their average, are:"<< endl;
    for (int j = 0; j < Scores; j++)
    {
        cout << fixed << setprecision(2) << setw(6) <<
(ScorePtr+j)->score << " " << (ScorePtr+j)->name << endl;
    }
    cout << "Average score:  " << setprecision(2) << fixed <<
Average << endl << endl;
    delete [] ScorePtr;
    ScorePtr = nullptr;
    return 0;
}
void sort(stu* score, int size)
{
    int start, minIndex;
    double minValue;
    // sorting in ascending order
    for (start = 0; start < (size - 1); start++)
    {
        minIndex = start;
        minValue = (score+start)->score;

        for(int index = start + 1; index < size; index++)
        {
            if ((score+index)->score < minValue)
            {
                minValue = (score+index)->score;
                minIndex = index;
            }
        }

        *(score+minIndex) = *(score+start);
        (score+start)->score = minValue;
    }
}

```

```

}
double average(stu* score, int numScores)
{
    double total = 0;
    for (int k = 0; k < numScores; k++)
        total += (score+k)->score;

    // Return the average score.
    return (total / numScores);
}

```

5. Pointer Rewrite

The following function uses reference variables as parameters. Rewrite the function so it uses pointers instead of reference variables, then demonstrate the function in a complete program.

```

int doSomething(int &x, int &y)
{
    int temp = x; x = y * 10;
    y = temp * 10;
    return x + y;
}

#include<bits/stdc++.h>
using namespace std;
void dosomething( int *x, int *y);
int main()
{
    int a=0,b=0;
    cin>>a>>b;
    dosomething(&a,&b);
    cout<<a<<" "<<b<<endl;
    return 0;
}

void dosomething( int *x, int *y)//pass the address of a b into
the function
{
    int temp=*x;
    *x=*y;
    *y=temp;
}

```

6. Case Study Modification #1

Modify Program 9-19 (the United Cause case study program) so it can be used with any set of donations. The program should dynamically allocate the donations array and ask the user to input its values.

```

//descending order
#include <iostream>
using namespace std;
void arrSelectSort(int *, int);
void showArray(int [], int);
int main()
{
    int *donations = nullptr;    // Points to a sorted array of
donations
    int *origin = nullptr;    // Points to an unsorted array of
donations
    int numDonations = 0;    // To hold the number
of donations
    cout << "How many donations would you like to enter? ";
    cin >> numDonations;
    while (numDonations <= 0)    // more than 0.
    {
        cout << endl;
        cout << "Please enter a positive number: ";
        cin >> numDonations;
    }
    donations = new int[numDonations];
    origin = new int[numDonations];
}

```

```

for (int k = 0; k < numDonations; k++)
{
    cout << "Enter a donation: ";
    cin >> donations[k];
    while (donations[k] <= 0)
    {
        cout << "Donations must be greater than 0." <<
endl;

        cout << "Enter a donation: ";
        cin >> donations[k];
    }
    origin[k] = donations[k];
}
arrSelectSort(donations, numDonations);
cout << "The donations, sorted in descending order
are:"<< endl;
showArray(donations, numDonations);
cout << "The donations, in their original order are: " <<
endl;
showArray(origin, numDonations);
delete [] donations;
delete [] origin;
donations = nullptr;
origin = nullptr;
return 0;
}

void arrSelectSort(int *values, int elems)
{
    int startScan, maxIndex, maxElem;
    for (startScan = 0; startScan < (elems - 1); startScan++)
    {
        maxIndex = startScan;
        maxElem = values[startScan];
        for(int index = startScan + 1; index < elems; index++)
        {
            if (values[index] > maxElem)
            {
                maxElem = values[index];
                maxIndex = index;
            }
        }
        values[maxIndex] = values[startScan];
        values[startScan] = maxElem;
    }
}

void showArray(int array[], int size)
{
    for (int count = 0; count < size; count++)
        cout << array[count] << " ";
    cout << endl;
}

```

7. Case Study Modification #2

Modify Program 9-19 (the United Cause case study program) so the arrptr array is sorted in descending order instead of ascending order.

```

//ascending order
#include <iostream>
using namespace std;
void arrSelectSort(int *, int);
void showArray(int [], int);
int main()
{
    int *donations = nullptr;    // Points to a sorted array of
donations
    int *origin = nullptr;    // Points to an unsorted array of
donations
    int numDonations = 0;    // To hold the number

```

of donations

```
cout << "How many donations would you like to enter? ";
cin >> numDonations;
while (numDonations <= 0) // more than 0.
{
    cout << endl;
    cout << "Please enter a positive number: ";
    cin >> numDonations;
}
donations = new int[numDonations];
origin = new int[numDonations];
for (int k = 0; k < numDonations; k++)
{
    cout << "Enter a donation: ";
    cin >> donations[k];
    while (donations[k] <= 0)
    {
        cout << "Donations must be greater than 0." <<
endl;
        cout << "Enter a donation: ";
        cin >> donations[k];
    }
    origin[k] = donations[k];
}
arrSelectSort(donations, numDonations);
cout << "The donations, sorted in ascending order are:" <<
endl;
showArray(donations, numDonations);
cout << "The donations, in their original order are: " <<
endl;
showArray(origin, numDonations);
delete [] donations;
delete [] origin;
donations = nullptr;
origin = nullptr;
return 0;
}

void arrSelectSort(int *values, int elems)//in ascending order
{
    int startScan, minIndex, minElem;
    for (startScan = 0; startScan < (elems - 1); startScan++)
    {
        minIndex = startScan;
        minElem = values[startScan];
        for(int index = startScan + 1; index < elems; index++)
        {
            if (values[index] < minElem)
            {
                minElem = values[index];
                minIndex = index;
            }
        }
        values[minIndex] = values[startScan];
        values[startScan] = minElem;
    }
}

void showArray(int array[], int size)
{
    for (int count = 0; count < size; count++)
        cout << array[count] << " ";
    cout << endl;
}
```

8. Mode Function

In statistics, the mode of a set of values is the value that occurs most often or with the greatest frequency. Write a function that accepts as arguments the following:

A) An array of integers

B) An integer that indicates the number of elements in the array. The function should determine the mode of the array. That is, it should determine which value in the array occurs most often. The mode is the value the function should return. If the array has no mode (none of the values occur more than once), the function should return -1. (Assume the array will always contain nonnegative values.)

Demonstrate your pointer prowess by using pointer notation instead of array notation in this function.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
struct statistics
```

```
{
```

```
    int number=-1;
```

```
    int frequency=0;
```

```
};
```

```
int ModeJudge(int *num, int length);
```

```
bool cmp(struct statistics a, struct statistics b)
```

```
{
```

```
    return a.frequency>b.frequency;
```

```
}
```

```
int main()
```

```
{
```

```
    int length=0;
```

```
    cin>>length;
```

```
    int *num=new int[length];
```

```
    for(int i=0;i<length;i++)
```

```
    {
```

```
        cin>>*(num+i);
```

```
    }
```

```
    int mode=ModeJudge(num,length);
```

```
    if(mode==-1) cout<<"No mode"<<endl;
```

```
    else cout<<"Mode: "<<mode<<endl;
```

```
    return 0;
```

```
}
```

```
int ModeJudge(int *num, int length)
```

```
{
```

```
    statistics *stat=new statistics[length];
```

```
    for(int i=0;i<length;i++)
```

```
    {
```

```
        for(int j=0;j<length;j++)//state the frequency of each
```

```
number
```

```
    {
```

```
        if((stat+j)->number== *(num+i)) //find
```

```
        {
```

```
            (stat+j)->frequency++;
```

```
            break;
```

```
        }
```

```
        else if((stat+j)->number==-1) //do not find
```

```
        {
```

```
            (stat+j)->number=*(num+i);
```

```
            (stat+j)->frequency=1;
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
    sort(stat,stat+length,cmp);
```

```
    if((stat+0)->frequency==(stat+1)->frequency) return -1; //
```

```
no mode
```

```
    else return (stat+0)->number;
```

```
}
```

9. Median Function

In statistics, when a set of values is sorted in ascending or descending order, its median is the middle value. If the set contains an even number of values, the median is the mean, or average, of the two middle values. Write a function that

accepts as arguments the following:

A) An array of integers

B) An integer that indicates the number of elements in the array. The function should determine the median of the array. This value should be returned as a double. (Assume the values in the array are already sorted.)

Demonstrate your pointer prowess by using pointer notation instead of array notation in this function.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
double median(int *array, int n);
```

```
int main()
```

```
{
    int num=0;
    cin>>num;
    int *arr= new int[num];
    for(int i=0;i<num;i++) cin>>*(arr+i);
    sort(arr,arr+num); //ascending order, creating condition
```

for our function

```
    double ans = median(arr,num);
    cout<<fixed<<setprecision(1)<<ans;
    delete [] arr; // free memory
    arr=nullptr; // avoid dangling pointer
    return 0;
}
```

```
double median(int *array, int n)
```

```
{
    if(n%2==0) //even
    {
        return (array[n/2-1]+array[n/2])/2.0;
    }
    else //odd
    {
        return array[n/2];
    }
}
```

10. Array Splitter

Write a function that accepts an int array and the array's size as arguments. The function should return a new array consisting of the values at even positions in the original array, that is, elements 0, 2, 4, 6, 8, and so on. The function should return a pointer to the new array. Demonstrate the function in a complete program.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void splitter(int *, int);
```

```
int main()
```

```
{
    int n;cin>>n;
    int *num=new int[n];
    for(int i=0;i<n;i++) cin>>*(num+i);
    splitter(num,n);
    delete [] num; // free memory
    num=nullptr; // avoid dangling pointer
    return 0;
}
```

```
void splitter(int *num, int n)
```

```
{
    for(int i=0;i<n;i+=2)
    {
        cout<<*(num+i)<<" ";
    }
    cout<<endl;
}
```

11. Array Joiner

Write a function that accepts two int arrays and their sizes as arguments. The function should return a new array consisting

of the values of both arrays. The elements of the second array should be joined after the elements of the first array. The function should return a pointer to the new array.

Demonstrate the function in a complete program.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int* connector(int *, int, int *, int);
```

```
int main()
```

```
{
    int n,m;cin>>n>>m;
    int *a=new int[n];
    int *b=new int[m];
    for(int i=0;i<n;i++) cin>>*(a+i);
    for(int i=0;i<m;i++) cin>>*(b+i);
    int *ans=connector(a,n,b,m);
    for(int i=0;i<n+m;i++) cout<<*(ans+i)<<" ";
    delete [] a;
    a=nullptr;
    delete [] b;
    b=nullptr;
    delete [] ans;
    ans=nullptr;
    return 0;
}
```

```
int* connector(int *arr1, int num1, int *arr2, int num2)
```

```
{
    int *res=new int[num1+num2];
    for(int i=0;i<num1;i++) *(res+i)=*(arr1+i);
    for(int i=0;i<num2;i++) *(res+num1+i)=*(arr2+i);
    return res;
}
```

12. Array Filter

Write a function that accepts three arguments: an int array, its size, and an int variable. The function should return a new array consisting of only those values in the original array that are a multiple of the third argument. For example, if the third argument is 7, only those values from the original array that are multiples of 7 should be in the new array. First calculate the size of the new array and then copy the correct values. The function should return a pointer to the new array.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int* selection(int* , int , int);
```

```
int sum=0;
```

```
int main()
```

```
{
    int n;cin>>n;
    int k;cin>>k;
    int *num= new int[n];
    for(int i=0;i<n;i++) cin>>*(num+i);
    int *selected=selection(num,n,k);
    for(int i=0;i<sum;i++) cout<<*(selected+i)<<" ";
    cout<<endl;
    delete [] num; // free memory
    delete [] selected; // free memory
    num=nullptr; // avoid dangling pointer
    selected=nullptr; // avoid dangling pointer
    return 0;
}
```

```
int* selection(int* arr, int n, int k)
```

```
{
    if (k == 0) {
        sum = 0;
        return new int[0];
    }
    sum = 0; // reset global counter each call
}
```

```

for(int i=0;i<n;i++)
{
    if(*(arr+i)%k==0)
    {
        sum++;
    }
}
int *selected=new int[sum];
int j = 0;
for(int i=0;i<n;i++)
{
    if(*(arr+i)%k==0)
    {
        *(selected+j)=*(arr+i);
        j++;
    }
}
return selected;
}

```

13. Movie Statistics

Write a program that can be used to gather statistical data about the number of movies college students see in a month. The program should perform the following steps:

A) Ask the user how many students were surveyed. An array of integers with this many elements should then be dynamically allocated.

B) Allow the user to enter the number of movies each student saw into the array.

C) Calculate and display the average, median, and mode of the values entered. (Use the functions you wrote in Programming Challenges 8 and 9 to calculate the median and mode.)

Input Validation: Do not accept negative numbers for input.

```

#include<bits/stdc++.h>
using namespace std;
double average(int *array, int n);
double median(int *array, int n);
double mode(int *array, int n);
int main()
{
    int num=0;cin>>num;
    int *MovieNum= new int[num];
    for(int i=0;i<num;i++)
    {
        int n=0;cin>>n;
        if(n<0)// input check
        {
            cout<<"Invalid input"<<endl;
            i--;
            continue;
        }
        else
            *(MovieNum+i)=n;
    }
    sort(MovieNum,MovieNum+num);//ascending order,
creating condition for our functions
    double avg = average(MovieNum,num);
    double med = median(MovieNum,num);
    double mod = mode(MovieNum,num);
    cout<<fixed<<setprecision(1)<<avg<<endl;
    cout<<fixed<<setprecision(1)<<med<<endl;
    cout<<fixed<<setprecision(1)<<mod<<endl;
    delete [] MovieNum;// free memory
    MovieNum=nullptr;// avoid dangling pointer
    return 0;
}
double average(int *array, int n)
{

```

```

double sum=0.0;
for(int i=0;i<n;i++)
{
    sum+=*(array+i);
}
return sum/n;
}
double median(int *array, int n)
{
    if(n%2==0)//even
    {
        return (array[n/2-1]+array[n/2])/2.0;
    }
    else//odd
    {
        return array[n/2];
    }
}
double mode(int *array, int n)
{
    int maxValue = -1, maxCount = 0;
    for (int i = 0; i < n; ++i) {
        int count = 0;
        for (int j = 0; j < n; ++j) {
            if (array[j] == array[i]) {
                count++;
            }
        }
        if (count > maxCount) {
            maxCount = count;
            maxValue = array[i];
        }
    }
    return maxValue;
}

```


Chapter 11 challenging problem

1. Movie Data

Create a structure named `MovieData` to store information about a movie:

Title

Director

Year Released, Running Time (in minutes). Create two

`MovieData` variables, store values in their members, and pass each one to a function that displays the movie information in a clearly formatted manner.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
struct MovieData
```

```
{
```

```
    string title;
```

```
    string director;
```

```
    int yearReleased;
```

```
    int runningTime;
```

```
};
```

```
void displayMovieInfo(MovieData *movie)
```

```
{
```

```
    cout<<"Title: "<<movie->title<<endl;
```

```
    cout<<"Director: "<<movie->director<<endl;
```

```
    cout<<"Year Released: "<<movie->yearReleased<<endl;
```

```
    cout<<"Running Time (in minutes):
```

```
"<<movie->runningTime<<endl;
```

```
}
```

```
int main()
```

```
{
```

```
    MovieData movie1={"Flipped","Rob Reiner",2010,90};
```

```
    MovieData movie2={"Zootopia","Byron Howard and Rich
```

```
Moore",2016,108};
```

```
    displayMovieInfo(&movie1);
```

```
    displayMovieInfo(&movie2);
```

```
    return 0;
```

```
}
```

2. Movie Profit

Modify the program written for Programming Challenge 1 (Movie Data) to include two additional members that hold the movie's production costs and first-year revenues. Modify the function that displays the movie data to display the title, director, release year, running time, and first year's profit or loss.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
struct MovieData
```

```
{
```

```
    string title;
```

```
    string director;
```

```
    int yearReleased;
```

```
    int runningTime;
```

```
    double cost;
```

```
    double revenue;
```

```
};
```

```
void displayMovieInfo(MovieData *movie)
```

```
{
```

```
    cout<<"Title: "<<movie->title<<endl;
```

```
    cout<<"Director: "<<movie->director<<endl;
```

```
    cout<<"Year Released: "<<movie->yearReleased<<endl;
```

```
    cout<<"Running Time (in minutes):
```

```
"<<movie->runningTime<<endl;
```

```
    cout<<"benifit: "<<(movie->revenue)-
```

```
(movie->cost)<<"$"<<endl;
```

```
}
```

```
int main()
```

```
{
```

```
    MovieData movie1={"Flipped","Rob
```

```
Reiner",2010,90,14000000,4300000};
```

```
    MovieData movie2={"Zootopia","Byron Howard and Rich
```

```
Moore",2016,108,150000000,1023784195};
```

```
    displayMovieInfo(&movie1);
```

```
    displayMovieInfo(&movie2);
```

```
    return 0;
```

```
}
```

3. Report on Earthquakes

The National Earthquake Information Center wants to record data on three earth quakes that have hit the country in the last two years. The department needs to store the following data about each earthquake:

Name of earthquake

Location of earthquake

Year

Magnitude in Richter scale

Fatalities

Write a program that declares a structure with fields to store the above data. The program should declare three variables of the above structure to represent the three earthquakes. The user should be asked to input the particulars of each earthquake. It should then display an analysis on the earthquakes. The particulars that should be shown in the analysis are:

- Total fatalities due to the three earthquakes.
- The average magnitude of the three earthquakes.
- Name and magnitude of the earthquake that had maximum magnitude.
- Name of the earthquake that caused maximum fatalities.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
struct earthquake
```

```
{
```

```
    string name;
```

```
    string location;
```

```
    int year;
```

```
    double magnitude;
```

```
    double fatalities;
```

```
};
```

```
int main()
```

```
{
```

```
    earthquake eq1,eq2,eq3;
```

```
    cout<<"Enter details of earthquake 1 (name, location, year, magnitude, fatalities): ";
```

```
    cin>>eq1.name>>eq1.location>>eq1.year>>eq1.magnitude>>eq1.fatalities;
```

```
    cout<<"Enter details of earthquake 2 (name, location, year, magnitude, fatalities): ";
```

```
    cin>>eq2.name>>eq2.location>>eq2.year>>eq2.magnitude>>eq2.fatalities;
```

```
    cout<<"Enter details of earthquake 3 (name, location, year, magnitude, fatalities): ";
```

```
    cin>>eq3.name>>eq3.location>>eq3.year>>eq3.magnitude>>eq3.fatalities;
```

```
    cout<<"\nEarthquake Details:\n";
```

```
    cout<<"total fatalities:
```

```
"<<(eq1.fatalities)+(eq2.fatalities)+(eq3.fatalities)<<endl;
```

```
    cout<<"Average Magnitude:
```

```
"<<((eq1.magnitude)+(eq2.magnitude)+(eq3.magnitude))/3<<endl;
```

```
    cout<<"maximum magnitude earthquake: ";
```

```
    if(eq1.magnitude>=eq2.magnitude &&
```

```
eq1.magnitude>=eq3.magnitude)
```

```
        cout<<eq1.name<<endl;
```

```

else if(eq2.magnitude>=eq1.magnitude &&
eq2.magnitude>=eq3.magnitude)
    cout<<eq2.name<<endl;
else
    cout<<eq3.name<<endl;
cout<<"maximum fatalities earthquake: ";
if(eq1.fatalities>=eq2.fatalities &&
eq1.fatalities>=eq3.fatalities)
    cout<<eq1.name<<endl;
else if(eq2.fatalities>=eq1.fatalities &&
eq2.fatalities>=eq3.fatalities)
    cout<<eq2.name<<endl;
else
    cout<<eq3.name<<endl;
return 0;
}

```

4. Weather Statistics

Write a program that uses a structure to store the following weather data for a particular month:

```

Total Rainfall
High Temperature
Low Temperature
Average Temperature

```

The program should have an array of 12 structures to hold weather data for an entire year. When the program runs, it should ask the user to enter data for each month. (The average temperature should be calculated.) Once the data are entered for all the months, the program should calculate and display the average monthly rainfall, the total rainfall for the year, the highest and lowest temperatures for the year (and the months they occurred in), and the average of all the monthly average temperatures. Input Validation: *Only accept temperatures within the range between -100 and +140 degrees Fahrenheit.*

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
struct Weather{
```

```

    double rainfall;
    double HighTemperature;
    double LowTemperature;
    double AverageTemperature;

```

```
};
```

```
int main()
```

```
{
```

```

    Weather month[12];
    double totalRainfall=0.0;
    double avgRainfall=0.0;
    double AverageTemperature=0.0;
    double highestTemperature=0;
    double lowestTemperature=0;
    int high=0,low=0;
    for(int i=0;i<12;i++)
    {

```

```

        cout<<"Enter the data of month "<<i+1<<endl;
        cout<<"Rainfall: ";
        cin>>month[i].rainfall;
        cout<<"High Temperature: ";
        cin>>month[i].HighTemperature;
        cout<<"Low Temperature: ";
        cin>>month[i].LowTemperature;
        if(month[i].HighTemperature< -100 ||

```

```

month[i].HighTemperature>140 || month[i].LowTemperature<
-100 || month[i].LowTemperature>140)
    {

```

```

        cout<<"Error: Temperature out of
range."<<endl;
        i--;

```

```
        continue;
```

```
    }
```

```

month[i].AverageTemperature=(month[i].HighTemperature+m
onth[i].LowTemperature)/2.0;
totalRainfall+=month[i].rainfall;

```

```

AverageTemperature+=month[i].AverageTemperature;
if(i==0)
{

```

```

    highestTemperature=month[i].HighTemperature;
    lowestTemperature=month[i].LowTemperature;
}
else
{

```

```

if(month[i].HighTemperature>highestTemperature)
{

```

```

    highestTemperature=month[i].HighTemperature;
    high=i;
}

```

```

if(month[i].LowTemperature<lowestTemperature)
{

```

```

    lowestTemperature=month[i].LowTemperature;
    low=i;
}

```

```

}
}

```

```
avgRainfall=totalRainfall/12.0;
```

```
AverageTemperature/=12.0;
```

```
cout<<fixed<<setprecision(2);
```

```

cout<<"Total Rainfall: "<<totalRainfall<<" inches"<<endl;
cout<<"Average Monthly Rainfall: "<<avgRainfall<<"

```

```
inches"<<endl;
```

```
    cout<<"Average of Average Monthly Temperatures:
```

```
"<<AverageTemperature<<" F"<<endl;
```

```
    cout<<"Highest Temperature: "<<highestTemperature<<"
```

```
F"<<" in month "<< high+1<<endl;
```

```
    cout<<"Lowest Temperature: "<<lowestTemperature<<"
```

```
F"<<" in month "<< low+1<<endl;
```

```
    return 0;
```

```
}
```

5. Weather Statistics Modification

Modify the program that you wrote for Programming Challenge 4 (weather statistics) so it defines an enumerated data type with enumerators for the months (JANUARY, FEBRUARY, so on). The program should use the enumerated type to step through the elements of the array.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
struct Weather{
```

```

    double rainfall;
    double HighTemperature;
    double LowTemperature;
    double AverageTemperature;

```

```
};
```

```
enum
```

```

Month{JANUARY=0,FEBRUARY,MARCH,APRIL,MAY,JUNE,JULY,A
UGUST,SEPTEMBER,OCTOBER,NOVEMBER,DECEMBER,MONTH
_COUNT};

```

```
int main()
```



```

{
    Weather month[MONTH_COUNT];
    double totalRainfall=0.0;
    double avgRainfall=0.0;
    double AverageTemperature=0.0;
    double highestTemperature=-200;
    double lowestTemperature=150;
    for(Month m=JANUARY; m<MONTH_COUNT;
m=Month(m+1))
    {
        int i = static_cast<int>(m);
        cout<<"Enter the data of month "<<i+1<<endl;
        cout<<"Rainfall: ";
        cin>>month[i].rainfall;
        cout<<"High Temperature: ";
        cin>>month[i].HighTemperature;
        cout<<"Low Temperature: ";
        cin>>month[i].LowTemperature;
        if(month[i].HighTemperature< -100 ||
month[i].HighTemperature>140 || month[i].LowTemperature<
-100 || month[i].LowTemperature>140)
        {
            cout<<"Error: Temperature out of
range."<<endl;
            m = Month(m-1);
            continue;
        }

        month[i].AverageTemperature=(month[i].HighTemperature+m
onth[i].LowTemperature)/2.0;
        totalRainfall+=month[i].rainfall;

        AverageTemperature+=month[i].AverageTemperature;
        if(i==0)
        {

            highestTemperature=month[i].HighTemperature;
            lowestTemperature=month[i].LowTemperature;
        }
        else
        {

            if(month[i].HighTemperature>highestTemperature)

            highestTemperature=month[i].HighTemperature;

            if(month[i].LowTemperature<lowestTemperature)

            lowestTemperature=month[i].LowTemperature;
        }

        avgRainfall=totalRainfall/static_cast<double>(MONTH_COUNT)
;

        AverageTemperature/=static_cast<double>(MONTH_COUNT);
        cout<<fixed<<setprecision(2);
        cout<<"Total Rainfall: "<<totalRainfall<<" inches"<<endl;
        cout<<"Average Monthly Rainfall: "<<avgRainfall<<"
inches"<<endl;
        cout<<"Average of Average Monthly Temperatures:
"<<AverageTemperature<<" F"<<endl;
        cout<<"Highest Temperature: "<<highestTemperature<<"
F"<<endl;
        cout<<"Lowest Temperature: "<<lowestTemperature<<"
F"<<endl;
        return 0;
    }
}

```

```

}
6. Popularity Score
Write a program that stores the following data about a quote
in a structure:
    The originator of the quote
    The quote itself
    The number of likes the quote has received
Write a function that, given an array of quotes and the length
of that array, calculates both the total number of likes and the
average number of likes. Demonstrate the function in a
program that asks the user to enter the data for seven quotes.
The program should show a table that lists only those quotes
that are more popular than the average quote and that shows
the percentage of likes each received.
Input Validation: Do not accept negative values for the number
of likes.
#include<bits/stdc++.h>
using namespace std;
struct texbib
{
    string originator;
    string quotation;
    int likes;
};
void display(texbib *citation,int n)
{
    int totallikes=0;
    double averlikes=0;
    for(int i=0;i<n;i++)
    {
        totallikes+=citation[i].likes;
    }
    averlikes=double(totallikes)/n;
    for(int i=0;i<n;i++)
    {
        if(citation[i].likes>averlikes)
        {
            cout<<"Citations with likes more than average
likes:\n";

            cout<<citation[i].originator<<"\n"<<citation[i].quotation<<"\n"
<<citation[i].likes<<"\n";
            cout<<"-----\n";
        }
    }
}
int main()
{
    int n=7;
    texbib citation[n];
    for(int i=0;i<n;i++)
    {
        cout<<"Enter originator, quotation and likes:\n";
        getline(cin,citation[i].originator);
        getline(cin,citation[i].quotation);
        int tmp;cin>>tmp;
        if(tmp<0)
        {
            i--;
            continue;
        }
        else
        {
            citation[i].likes=tmp;
            cin.ignore();
        }
    }
}

```

```
}
```

7. Computer Configurations

Write a program that uses a structure to store the following data about a computer:

- The clock speed of the processor
- The number of CPU cores
- The amount of RAM The size of the hard disk
- The benchmark score (higher is faster)
- The price of the computer

The program should have a menu-driven interface with the options to add a new computer configuration, to change the price of an existing configuration, and to show the full data of all computers. The user selects a configuration by entering its number.

Input Validation: Do not accept negative values for the benchmark score or the price. When entering the number of a configuration, accept only existing numbers.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <limits>
```

```
#include <iomanip>
```

```
#include <string>
```

```
using namespace std;
```

```
struct ComputerConfig
```

```
{
    double clockGHz;    // CPU clock speed
    int cpuCores;       // Number of CPU cores
    int ramGB;          // RAM size
    int diskGB;         // Hard disk size
    double benchmark;   // Benchmark score (higher is
                        // faster)
    double price;       // Price
};
```

```
void clear_input()
```

```
{
    cin.clear();
    cin.ignore(1000, '\n');
}
```

```
double read_ND(const string &prompt)
```

```
{
    while (true)
    {
        cout << prompt;
        double v;
        if (cin >> v)
        {
            if (v >= 0.0) return v;
            cout << "Value must be non-negative."<<endl;
        }
        else
        {
            cout << "Invalid input. "<<endl;
        }
        clear_input();
    }
}
```

```
int read_PI(const string &prompt)
```

```
{
    while (true)
    {
```

```
    cout << prompt;
```

```
    int v;
```

```
    if (cin >> v)
```

```
    {
```

```
        if (v > 0) return v;
```

```
        cout << "Value must be a positive
```

```
integer."<<endl;
```

```
    }
```

```
    else
```

```
    {
```

```
        cout << "Invalid input. "<<endl;
```

```
    }
```

```
    clear_input();
```

```
}
```

```
}
```

```
int read_NI(const string &prompt)
```

```
{
```

```
    while (true)
```

```
    {
```

```
        cout << prompt;
```

```
        int v;
```

```
        if (cin >> v)
```

```
        {
```

```
            if (v >= 0) return v;
```

```
            cout << "Value must be a non-negative
```

```
integer."<<endl;
```

```
        }
```

```
        else
```

```
        {
```

```
            cout << "Invalid input. "<<endl;
```

```
        }
```

```
        clear_input();
```

```
    }
```

```
}
```

```
void printConfig(const ComputerConfig &c, int indexDisplay)
```

```
{
```

```
    cout << "No.: " << indexDisplay << endl;
```

```
    cout << fixed << setprecision(2);
```

```
    cout << "    Clock (GHz): " << c.clockGHz << endl;
```

```
    cout << "    CPU Cores: " << c.cpuCores << endl;
```

```
    cout << "    RAM (GB): " << c.ramGB << endl;
```

```
    cout << "    Disk (GB): " << c.diskGB << endl;
```

```
    cout << "    Benchmark: " << c.benchmark << endl;
```

```
    cout << "    Price: " << c.price << endl;
```

```
}
```

```
void printAll(const vector<ComputerConfig> &list)
```

```
{
```

```
    if (list.empty())
```

```
    {
```

```
        cout << "No configurations yet." << endl;
```

```
        return;
```

```
    }
```

```
    cout << "-- All Computer Configurations --" << endl;
```

```
    for (size_t i = 0; i < list.size(); ++i)
```

```
    {
```

```
        printConfig(list[i], static_cast<int>(i) + 1);
```

```
        if (i + 1 < list.size()) cout << "-----" <<
```

```
endl;
```

```
    }
```

```
}
```

```
void addConfig(vector<ComputerConfig> &list)
```

```
{
```

```

    ComputerConfig c;
    c.clockGHz = read_ND("Enter clock speed (GHz): ");
    c.cpuCores = read_PI("Enter number of CPU cores: ");
    c.ramGB = read_PI("Enter RAM size (GB): ");
    c.diskGB = read_PI("Enter hard disk size (GB): ");
    c.benchmark = read_ND("Enter benchmark score: ");
    c.price = read_ND("Enter price: ");
    list.push_back(c);
    cout << "Added. Total count: " << list.size() << endl;
}

void changePrice(vector<ComputerConfig> &list)
{
    if (list.empty())
    {
        cout << "List is empty. Cannot change price." << endl;
        return;
    }
    cout << "There are " << list.size() << " configurations." <<
endl;
    int num;
    while (true)
    {
        num = read_PI("Enter configuration number to
change price (1-" + to_string(list.size()) + "): ");
        if (num >= 1 && static_cast<size_t>(num) <=
list.size()) break;
        cout << "Number does not exist." << endl;
    }
    double newPrice = read_ND("Enter new price: ");
    list[static_cast<size_t>(num) - 1].price = newPrice;
    cout << "Updated price for configuration " << num << "."
<< endl;
}

void showOne(const vector<ComputerConfig> &list)
{
    if (list.empty())
    {
        cout << "List is empty." << endl;
        return;
    }
    int num;
    while (true)
    {
        num = read_PI("Enter configuration number to show
(1-" + to_string(list.size()) + "): ");
        if (num >= 1 && static_cast<size_t>(num) <=
list.size()) break;
        cout << "Number does not exist." << endl;
    }
    printConfig(list[static_cast<size_t>(num) - 1], num);
}

void printMenu()
{
    cout << endl;
    cout << "==== Menu =====< endl;
    cout << "1) Add new configuration" << endl;
    cout << "2) Change price of existing configuration" <<
endl;
    cout << "3) Show all configurations" << endl;
    cout << "4) Show a configuration by number" << endl;
    cout << "5) Exit" << endl;
}

int main()

```

```

{
    vector<ComputerConfig> configs;

    while (true)
    {
        printMenu();
        cout << "Choose (1-5): ";
        int choice;
        if (!(cin >> choice))
        {
            cout << "Invalid input. Please enter an integer."
<< endl;
            clear_input();
            continue;
        }

        switch (choice)
        {
            case 1:
                addConfig(configs);
                break;
            case 2:
                changePrice(configs);
                break;
            case 3:
                printAll(configs);
                break;
            case 4:
                showOne(configs);
                break;
            case 5:
                cout << "Exit." << endl;
                return 0;
            default:
                cout << "Invalid option." << endl;
                break;
        }
    }
}

```

8. Search Function for Computers

Add a function to Programming Challenge 7 (Computer Configurations) that allows the user to search for a computer that meets a minimum specification and that remains within budget. The function should accept a minimum value for the benchmark score and a maximum value for the price as arguments, and display all suitable configurations. If no configurations match, a message saying so should be displayed.

```

#include <iostream>
#include <vector>
#include <limits>
#include <iomanip>
#include <string>
using namespace std;

```

```

struct ComputerConfig
{

```

```

    double clockGHz;        // CPU clock speed
    int cpuCores;           // Number of CPU cores
    int ramGB;              // RAM size
    int diskGB;             // Hard disk size
    double benchmark;       // Benchmark score (higher is
faster)
    double price;           // Price
};

```

```

void clear_input()
{
    cin.clear();
    cin.ignore(1000, '\n');
}

double read_ND(const string &prompt)
{
    while (true)
    {
        cout << prompt;
        double v;
        if (cin >> v)
        {
            if (v >= 0.0) return v;
            cout << "Value must be non-negative."<<endl;
        }
        else
        {
            cout << "Invalid input. "<<endl;
        }
        clear_input();
    }
}

```

```

int read_PI(const string &prompt)
{
    while (true)
    {
        cout << prompt;
        int v;
        if (cin >> v)
        {
            if (v > 0) return v;
            cout << "Value must be a positive
integer."<<endl;
        }
        else
        {
            cout << "Invalid input. "<<endl;
        }
        clear_input();
    }
}

```

```

int read_NI(const string &prompt)
{
    while (true)
    {
        cout << prompt;
        int v;
        if (cin >> v)
        {
            if (v >= 0) return v;
            cout << "Value must be a non-negative
integer."<<endl;
        }
        else
        {
            cout << "Invalid input. "<<endl;
        }
        clear_input();
    }
}

```

```

void printConfig(const ComputerConfig &c, int indexDisplay)
{
    cout << "No.: " << indexDisplay << endl;
    cout << fixed << setprecision(2);
    cout << "    Clock (GHz): " << c.clockGHz << endl;
    cout << "    CPU Cores: " << c.cpuCores << endl;
    cout << "    RAM (GB): " << c.ramGB << endl;
    cout << "    Disk (GB): " << c.diskGB << endl;
    cout << "    Benchmark: " << c.benchmark << endl;
    cout << "    Price: " << c.price << endl;
}

```

```

void printAll(const vector<ComputerConfig> &list)
{
    if (list.empty())
    {
        cout << "No configurations yet." << endl;
        return;
    }
    cout << "-- All Computer Configurations --" << endl;
    for (size_t i = 0; i < list.size(); ++i)
    {
        printConfig(list[i], static_cast<int>(i) + 1);
        if (i + 1 < list.size()) cout << "-----" <<
endl;
    }
}

```

```

void addConfig(vector<ComputerConfig> &list)
{
    ComputerConfig c;
    c.clockGHz = read_ND("Enter clock speed (GHz): ");
    c.cpuCores = read_PI("Enter number of CPU cores: ");
    c.ramGB = read_PI("Enter RAM size (GB): ");
    c.diskGB = read_PI("Enter hard disk size (GB): ");
    c.benchmark = read_ND("Enter benchmark score: ");
    c.price = read_ND("Enter price: ");
    list.push_back(c);
    cout << "Added. Total count: " << list.size() << endl;
}

```

```

void changePrice(vector<ComputerConfig> &list)
{
    if (list.empty())
    {
        cout << "List is empty. Cannot change price." << endl;
        return;
    }
    cout << "There are " << list.size() << " configurations." <<
endl;
    int num;
    while (true)
    {
        num = read_PI("Enter configuration number to
change price (1-" + to_string(list.size()) + "): ");
        if (num >= 1 && static_cast<size_t>(num) <=
list.size()) break;
        cout << "Number does not exist." << endl;
    }
    double newPrice = read_ND("Enter new price: ");
    list[static_cast<size_t>(num) - 1].price = newPrice;
    cout << "Updated price for configuration " << num << "."
<< endl;
}

```

```

void showOne(const vector<ComputerConfig> &list)

```

```

{
    if (list.empty())
    {
        cout << "List is empty." << endl;
        return;
    }
    int num;
    while (true)
    {
        num = read_PI("Enter configuration number to show
(1-" + to_string(list.size()) + "): ");
        if (num >= 1 && static_cast<size_t>(num) <=
list.size()) break;
        cout << "Number does not exist." << endl;
    }
    printConfig(list[static_cast<size_t>(num) - 1], num);
}

// Search function: find configurations meeting min benchmark
and within max budget
void searchByBenchmarkAndPrice(const
vector<ComputerConfig> &list)
{
    if (list.empty())
    {
        cout << "No configurations yet." << endl;
        return;
    }

    double minBenchmark = read_ND("Enter minimum
benchmark score: ");
    double maxPrice = read_ND("Enter maximum price: ");

    vector<int> matchedIndices;
    for (size_t i = 0; i < list.size(); ++i)
    {
        if (list[i].benchmark >= minBenchmark && list[i].price
<= maxPrice)
        {
            matchedIndices.push_back(static_cast<int>(i));
        }
    }

    if (matchedIndices.empty())
    {
        cout << "No configurations match the given criteria."
<< endl;
        return;
    }

    cout << "-- Suitable Configurations --" << endl;
    for (size_t j = 0; j < matchedIndices.size(); ++j)
    {
        int idx = matchedIndices[j];
        printConfig(list[static_cast<size_t>(idx)], idx + 1);
        if (j + 1 < matchedIndices.size()) cout << "-----
-----" << endl;
    }
}

void printMenu()
{
    cout << endl;
    cout << "===== Menu =====" << endl;
    cout << "1) Add new configuration" << endl;
    cout << "2) Change price of existing configuration" <<
endl;

```

```

    cout << "3) Show all configurations" << endl;
    cout << "4) Show a configuration by number" << endl;
    cout << "5) Search by benchmark(min) and price(max)" <<
endl;
    cout << "6) Exit" << endl;
}

int main()
{
    vector<ComputerConfig> configs;

    while (true)
    {
        printMenu();
        cout << "Choose (1-6): ";
        int choice;
        if (!(cin >> choice))
        {
            cout << "Invalid input. Please enter an integer."
<< endl;
            clear_input();
            continue;
        }

        switch (choice)
        {
            case 1:
                addConfig(configs);
                break;
            case 2:
                changePrice(configs);
                break;
            case 3:
                printAll(configs);
                break;
            case 4:
                showOne(configs);
                break;
            case 5:
                searchByBenchmarkAndPrice(configs);
                break;
            case 6:
                cout << "Exit." << endl;
                return 0;
            default:
                cout << "Invalid option." << endl;
                break;
        }
    }
}

```

9. Speakers' Bureau

Write a program that keeps track of a speakers' bureau. The program should use a structure to store the following data about a speaker: Name Telephone Number Speaking Topic Fee Required The program should use an array of at least 10 structures. It should let the user enter data into the array, change the contents of any element, and display all the data stored in the array. The program should have a menu-driven user interface.

Input Validation: When the data for a new speaker is entered, be sure the user enters data for all the fields. No negative amounts should be entered for a speaker's fee.

```

#include <iostream>
#include <iomanip>
#include <limits>
#include <string>
using namespace std;

```

```

struct Speaker {
    string name;
    string phone;
    string topic;
    double fee = 0.0;
    bool occupied = false;
};

void clear_input() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

int readIndexInRange(const string &prompt, int lo, int hi) {
    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) {
            clear_input();
            if (v >= lo && v <= hi) return v;
            cout << "Value out of range." << endl;
        } else {
            cout << "Invalid input." << endl;
            clear_input();
        }
    }
}

double readNonNegativeDouble(const string &prompt) {
    while (true) {
        cout << prompt;
        double v;
        if (cin >> v) {
            clear_input();
            if (v >= 0.0) return v;
            cout << "Fee must be non-negative." << endl;
        } else {
            cout << "Invalid number." << endl;
            clear_input();
        }
    }
}

string readNonEmptyLine(const string &prompt) {
    while (true) {
        cout << prompt;
        string s;
        getline(cin, s);
        if (!s.empty()) return s;
        cout << "This field cannot be empty." << endl;
    }
}

void printOne(const Speaker &s, int idxDisplay) {
    cout << "No.: " << idxDisplay << endl;
    if (!s.occupied) {
        cout << "    <empty>" << endl;
        return;
    }
    cout << fixed << setprecision(2);
    cout << "    Name: " << s.name << endl;
    cout << "    Telephone: " << s.phone << endl;
    cout << "    Topic: " << s.topic << endl;
    cout << "    Fee: " << s.fee << endl;
}

```

```

void enterOrEdit(Speaker arr[], int n) {
    int idx = readIndexInRange("Enter slot to fill/edit (1-" +
to_string(n) + "): ", 1, n) - 1;
    if (arr[idx].occupied) {
        cout << "This slot currently has data, it will be
overwritten." << endl;
    }

    string name = readNonEmptyLine("Enter name: ");
    string phone = readNonEmptyLine("Enter telephone
number: ");
    string topic = readNonEmptyLine("Enter speaking topic:
");
    double fee = readNonNegativeDouble("Enter fee
required: ");

    arr[idx].name = name;
    arr[idx].phone = phone;
    arr[idx].topic = topic;
    arr[idx].fee = fee;
    arr[idx].occupied = true;
    cout << "Saved to slot " << (idx + 1) << "." << endl;
}

void editExisting(Speaker arr[], int n) {
    int idx = readIndexInRange("Enter speaker number to edit
(1-" + to_string(n) + "): ", 1, n) - 1;
    if (!arr[idx].occupied) {
        cout << "No data in this slot. Use 'Enter new' to add."
<< endl;
        return;
    }
    cout << "Re-enter all fields for this speaker:" << endl;
    arr[idx].name = readNonEmptyLine("Enter name: ");
    arr[idx].phone = readNonEmptyLine("Enter telephone
number: ");
    arr[idx].topic = readNonEmptyLine("Enter speaking topic:
");
    arr[idx].fee = readNonNegativeDouble("Enter fee
required: ");
    arr[idx].occupied = true;
    cout << "Updated speaker " << (idx + 1) << "." << endl;
}

void showAll(const Speaker arr[], int n) {
    cout << "-- All Speakers --" << endl;
    for (int i = 0; i < n; ++i) {
        printOne(arr[i], i + 1);
        if (i + 1 < n) cout << "-----" << endl;
    }
}

void showOneByNumber(const Speaker arr[], int n) {
    int idx = readIndexInRange("Enter number to show (1-" +
to_string(n) + "): ", 1, n) - 1;
    printOne(arr[idx], idx + 1);
}

void printMenu() {
    cout << endl;
    cout << "===== Speakers' Bureau =====" << endl;
    cout << "1) Enter new speaker" << endl;
    cout << "2) Edit a speaker" << endl;
    cout << "3) Show all speakers" << endl;
    cout << "4) Show a speaker by number" << endl;
    cout << "5) Exit" << endl;
}

```



```

int main() {
    const int N = 10; // at least 10
    Speaker bureau[N];

    while (true) {
        printMenu();
        cout << "Choose (1-5): ";
        int choice;
        if (cin >> choice) {
            clear_input();
        } else {
            cout << "Invalid input. Please enter an integer."
            << endl;
            clear_input();
            continue;
        }

        switch (choice) {
            case 1:
                enterOrEdit(bureau, N);
                break;
            case 2:
                editExisting(bureau, N);
                break;
            case 3:
                showAll(bureau, N);
                break;
            case 4:
                showOneByNumber(bureau, N);
                break;
            case 5:
                cout << "Exit." << endl;
                return 0;
            default:
                cout << "Invalid option." << endl;
                break;
        }
    }
}

```

10. Search Function for the Speakers' Bureau Program

Add a function to Programming Challenge 9 (Speakers' Bureau) that allows the user to search for a speaker on a particular topic. It should accept a key word as an argument then search the array for a structure with that key word in the Speaking Topic field. All structures that match should be displayed. If no structure matches, a message saying so should be displayed.

```

#include <iostream>
#include <iomanip>
#include <limits>
#include <string>
using namespace std;

```

```

struct Speaker {
    string name;
    string phone;
    string topic;
    double fee = 0.0;
    bool occupied = false;
};

```

```

void clear_input() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

```

```

int readIndexInRange(const string &prompt, int lo, int hi) {

```

```

    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) {
            clear_input();
            if (v >= lo && v <= hi) return v;
            cout << "Value out of range." << endl;
        } else {
            cout << "Invalid input." << endl;
            clear_input();
        }
    }
}

double readNonNegativeDouble(const string &prompt) {
    while (true) {
        cout << prompt;
        double v;
        if (cin >> v) {
            clear_input();
            if (v >= 0.0) return v;
            cout << "Fee must be non-negative." << endl;
        } else {
            cout << "Invalid number." << endl;
            clear_input();
        }
    }
}

string readNonEmptyLine(const string &prompt) {
    while (true) {
        cout << prompt;
        string s;
        getline(cin, s);
        if (!s.empty()) return s;
        cout << "This field cannot be empty." << endl;
    }
}

static string toLowerCopy(string s){
    for(char &ch: s){ ch =
static_cast<char>(tolower(static_cast<unsigned char>(ch))); }
    return s;
}

void printOne(const Speaker &s, int idxDisplay) {
    cout << "No.: " << idxDisplay << endl;
    if (!s.occupied) {
        cout << "    <empty>" << endl;
        return;
    }
    cout << fixed << setprecision(2);
    cout << "    Name: " << s.name << endl;
    cout << "    Telephone: " << s.phone << endl;
    cout << "    Topic: " << s.topic << endl;
    cout << "    Fee: " << s.fee << endl;
}

void searchByTopic(const Speaker arr[], int n) {
    string key = readNonEmptyLine("Enter topic keyword to
search: ");
    string keyLower = toLowerCopy(key);
    bool found = false;
    cout << "-- Search Results --" << endl;
    for (int i = 0; i < n; ++i) {
        if (!arr[i].occupied) continue;
        string topicLower = toLowerCopy(arr[i].topic);

```

```

        if (topicLower.find(keyLower) != string::npos) {
            found = true;
            printOne(arr[i], i + 1);
            if (i + 1 < n) cout << "-----" << endl;
        }
    }
    if (!found) {
        cout << "No speaker matches the given keyword." <<
endl;
    }
}

void enterOrEdit(Speaker arr[], int n) {
    int idx = readIndexInRange("Enter slot to fill/edit (1-" +
to_string(n) + "): ", 1, n) - 1;
    if (arr[idx].occupied) {
        cout << "This slot currently has data, it will be
overwritten." << endl;
    }

    string name = readNonEmptyLine("Enter name: ");
    string phone = readNonEmptyLine("Enter telephone
number: ");
    string topic = readNonEmptyLine("Enter speaking topic:
");
    double fee = readNonNegativeDouble("Enter fee
required: ");

    arr[idx].name = name;
    arr[idx].phone = phone;
    arr[idx].topic = topic;
    arr[idx].fee = fee;
    arr[idx].occupied = true;
    cout << "Saved to slot " << (idx + 1) << "." << endl;
}

void editExisting(Speaker arr[], int n) {
    int idx = readIndexInRange("Enter speaker number to edit
(1-" + to_string(n) + "): ", 1, n) - 1;
    if (!arr[idx].occupied) {
        cout << "No data in this slot. Use 'Enter new' to add."
<< endl;
        return;
    }
    cout << "Re-enter all fields for this speaker:" << endl;
    arr[idx].name = readNonEmptyLine("Enter name: ");
    arr[idx].phone = readNonEmptyLine("Enter telephone
number: ");
    arr[idx].topic = readNonEmptyLine("Enter speaking topic:
");
    arr[idx].fee = readNonNegativeDouble("Enter fee
required: ");
    arr[idx].occupied = true;
    cout << "Updated speaker " << (idx + 1) << "." << endl;
}

void showAll(const Speaker arr[], int n) {
    cout << "-- All Speakers --" << endl;
    for (int i = 0; i < n; ++i) {
        printOne(arr[i], i + 1);
        if (i + 1 < n) cout << "-----" << endl;
    }
}

void showOneByNumber(const Speaker arr[], int n) {
    int idx = readIndexInRange("Enter number to show (1-" +
to_string(n) + "): ", 1, n) - 1;

```

```

        printOne(arr[idx], idx + 1);
    }

void printMenu() {
    cout << endl;
    cout << "==== Speakers' Bureau =====" << endl;
    cout << "1) Enter new speaker" << endl;
    cout << "2) Edit a speaker" << endl;
    cout << "3) Show all speakers" << endl;
    cout << "4) Show a speaker by number" << endl;
    cout << "5) Search speakers by topic keyword" << endl;
    cout << "6) Exit" << endl;
}

int main() {
    const int N = 10; // at least 10
    Speaker bureau[N];

    while (true) {
        printMenu();
        cout << "Choose (1-6): ";
        int choice;
        if (cin >> choice) {
            clear_input();
        } else {
            cout << "Invalid input. Please enter an integer."
<< endl;
            clear_input();
            continue;
        }

        switch (choice) {
            case 1:
                enterOrEdit(bureau, N);
                break;
            case 2:
                editExisting(bureau, N);
                break;
            case 3:
                showAll(bureau, N);
                break;
            case 4:
                showOneByNumber(bureau, N);
                break;
            case 5:
                searchByTopic(bureau, N);
                break;
            case 6:
                cout << "Exit." << endl;
                return 0;
            default:
                cout << "Invalid option." << endl;
                break;
        }
    }
}

```

11. Comparison of Date and Time

Write a program that has a `dateTime` structure to store the following particulars about date and time:

```

Year
Month
Date
Hours
Minutes
Seconds

```

The program should declare two `dateTime` variables of the above structure and ask the user to input data on the date and

time of two different instances. The program should pass these structure variables as arguments to a function. The function should determine the date (and not time) that comes earlier between the two arguments. If both dates are same, the function displays a message "Identical dates" and returns any of the arguments; otherwise it should return the date that comes earlier.

Input Validation: Do not accept negative values for year. Accept values for month only within the range 1 – 12. Accept hours between 0 and 23, minutes between 0 and 59, and seconds between 0 and 59.

```
#include <iostream>
#include <limits>
#include <string>
using namespace std;
```

```
struct dateTime {
    int year;
    int month;
    int date;    // day of month
    int hours;
    int minutes;
    int seconds;
};
```

```
void clear_input() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
```

```
int readIntAny(const string &prompt) {
    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) { clear_input(); return v; }
        cout << "Invalid input." << endl;
        clear_input();
    }
}
```

```
int readNonNegativeInt(const string &prompt) {
    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) {
            clear_input();
            if (v >= 0) return v; // year must be non-negative
            cout << "Value must be non-negative." << endl;
        } else {
            cout << "Invalid input." << endl;
            clear_input();
        }
    }
}
```

```
int readIntInRange(const string &prompt, int lo, int hi) {
    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) {
            clear_input();
            if (v >= lo && v <= hi) return v;
            cout << "Value out of range." << endl;
        } else {
            cout << "Invalid input." << endl;
            clear_input();
        }
    }
}
```

```
    }
}

dateTime readDateTime(const string &title) {
    cout << "-- Enter " << title << " --" << endl;
    dateTime dt{};
    dt.year = readNonNegativeInt("Year (>=0): ");
    dt.month = readIntInRange("Month (1-12): ", 1, 12);
    dt.date = readIntAny("Date (day): ");
    dt.hours = readIntInRange("Hours (0-23): ", 0, 23);
    dt.minutes = readIntInRange("Minutes (0-59): ", 0, 59);
    dt.seconds = readIntInRange("Seconds (0-59): ", 0, 59);
    return dt;
}

// Compare only date part (year, month, date).
// If identical, print message and return any argument (first).
dateTime earlierDate(const dateTime &a, const dateTime &b) {
    if (a.year != b.year) return (a.year < b.year) ? a : b;
    if (a.month != b.month) return (a.month < b.month) ? a : b;
    if (a.date != b.date) return (a.date < b.date) ? a : b;
    cout << "Identical dates" << endl;
    return a;
}

int main() {
    dateTime d1 = readDateTime("first date & time");
    dateTime d2 = readDateTime("second date & time");

    dateTime e = earlierDate(d1, d2);
    cout << "Earlier date: " << e.year << "-" << e.month << "-"
    << e.date << endl;
    return 0;
}
```

12: Restaurant Top Dishes

A brochure for a region popular with tourists gives all the restaurants in the region the opportunity to advertise their most popular dishes. Write a program that uses a structure to store the following data:

Member Name Description

Name	Restaurant name
Description	Short description of the restaurant
Menus	Pointer to an array of three menu items
AveragePrice	The average price of the items

VegetarianOptions A Boolean indicating whether vegetarian dishes are available

The Menus member in this structure should point to an array of length 3 that will hold different menu items. The MenuItem is itself a simple structure that holds three members, the name of the dish (a string), a link to a picture of the dish (a string), and the price (a double).

The program should keep a list of restaurants and their menus. It should first ask the user for the number of restaurants. It should then ask the user to input all data. For each restaurant, it should first ask the name and the description of the restaurant, and whether it offers vegetarian dishes. Next the user must input the data of the three most popular menu items. The average price should be calculated and stored in the corresponding member of each restaurant structure.

Once everything is calculated, the program should ask the user for a price range (mini mum and maximum price) and display all the restaurants whose average price falls in that price range.

Input Validation: Be sure all the data for each restaurant and menu item is entered. Do not accept negative numbers for the price of a menu item.

```
#include <iostream>
#include <limits>
```

```

#include <string>
#include <vector>
using namespace std;

struct MenuItem {
    string name;        // dish name
    string picture;     // link to a picture of the dish
    double price = 0.0; // non-negative
};

struct Restaurant {
    string name;        // Restaurant name
    string description; // Short description
    MenuItem* menus = nullptr; // Pointer to array of length
    3
    double averagePrice = 0.0; // Average price of the 3 items
    bool vegetarianOptions = false; // Whether vegetarian
    dishes are available
};

void clear_input() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

int readPositiveInt(const string &prompt) {
    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) {
            clear_input();
            if (v > 0) return v;
            cout << "Value must be a positive integer." <<
endl;
        } else {
            cout << "Invalid input." << endl;
            clear_input();
        }
    }
}

double readNonNegativeDouble(const string &prompt) {
    while (true) {
        cout << prompt;
        double v;
        if (cin >> v) {
            clear_input();
            if (v >= 0.0) return v;
            cout << "Price must be non-negative." << endl;
        } else {
            cout << "Invalid number." << endl;
            clear_input();
        }
    }
}

string readNonEmptyLine(const string &prompt) {
    while (true) {
        cout << prompt;
        string s;
        getline(cin, s);
        if (!s.empty()) return s;
        cout << "This field cannot be empty." << endl;
    }
}

bool readYesNo(const string &prompt) {

```

```

while (true) {
    cout << prompt << " (y/n): ";
    string s; getline(cin, s);
    if (s.size()) {
        char c =
static_cast<char>(tolower(static_cast<unsigned char>(s[0])));
        if (c == 'y') return true;
        if (c == 'n') return false;
    }
    cout << "Please enter 'y' or 'n'." << endl;
}

int main() {
    cout << "-- Restaurant Top Dishes --" << endl;
    int n = readPositiveInt("Enter number of restaurants: ");
    vector<Restaurant> rs(n);

    for (int i = 0; i < n; ++i) {
        cout << "\nRestaurant #" << (i + 1) << endl;
        rs[i].name = readNonEmptyLine("Name: ");
        rs[i].description = readNonEmptyLine("Short
description: ");
        rs[i].vegetarianOptions = readYesNo("Vegetarian
options available?");

        rs[i].menus = new MenuItem[3];
        double sum = 0.0;
        for (int j = 0; j < 3; ++j) {
            cout << "    Menu item #" << (j + 1) << endl;
            rs[i].menus[j].name = readNonEmptyLine("
Dish name: ");
            rs[i].menus[j].picture = readNonEmptyLine("
Picture link: ");
            rs[i].menus[j].price = readNonNegativeDouble("
Price: ");
            sum += rs[i].menus[j].price;
        }
        rs[i].averagePrice = sum / 3.0;

        cout << "\n-- Filter by average price range --" << endl;
        double minP = readNonNegativeDouble("Minimum
average price: ");
        double maxP = readNonNegativeDouble("Maximum
average price: ");
        while (maxP < minP) {
            cout << "Maximum cannot be less than minimum.
Please re-enter." << endl;
            minP = readNonNegativeDouble("Minimum average
price: ");
            maxP = readNonNegativeDouble("Maximum average
price: ");
        }

        cout << "\nRestaurants within the range [" << minP << ", "
<< maxP << "]:" << endl;
        bool any = false;
        for (int i = 0; i < n; ++i) {
            if (rs[i].averagePrice >= minP && rs[i].averagePrice
<= maxP) {
                any = true;
                cout << "- " << rs[i].name << " | avg: " <<
rs[i].averagePrice << endl;
            }
        }
        if (!any) cout << "No restaurants found in the given price

```

```
range." << endl;

// clean up allocated menus
for (int i = 0; i < n; ++i) {
    delete[] rs[i].menus;
    rs[i].menus = nullptr;
}
return 0;
}
```

13. Full Restaurant Menu

Modify the solution of Programming Challenge 12 (Restaurant Top Dishes) such that each restaurant can store its full menu. The Restaurant structure should also be expanded with the following data:

Member Name	Description
MenuSize	The size of the menu
Minimum	The price of the cheapest menu item
Maximum	The price of the most expensive menu item

The Menus member in the restaurant structure should be modified such that it now points to a dynamically allocated array that will hold the different menu items. Because the Menus member is a dynamically allocated array, the number of menu items can now be different for each restaurant. The MenuItem structure should be expanded to hold a member variable that indicates whether a dish is vegetarian. The program should still first ask the user for the number of restaurants and dynamically allocate an array of structures to store all the restaurants. For each restaurant, it should now only ask the name and the description of the restaurant. Next the program should ask for the number of items on the menu. It should allocate an array of structures for the specified number of menu items and let the user enter all the menu items for that restaurant. Instead of only calculating the average price, the minimum and maximum price should also be calculated and stored in the corresponding members of each structure. The Boolean that indicates whether the restaurant offers vegetarian options can be derived from the menu items of the restaurant. A restaurant offers vegetarian options if at least one menu item is vegetarian. Once everything has been calculated, the program should now ask the user for a single price and display all the restaurants where the price chosen by the user is between the minimum and maximum price of the restaurant.

Input Validation: Be sure all the data for each restaurant and menu item is entered. Do not accept negative numbers for the price of a menu item.

```
#include <iostream>
#include <limits>
#include <string>
#include <cctype>
using namespace std;
```

```
struct MenuItem {
    string name; // dish name
    string picture; // link to a picture of the dish
    double price = 0.0; // non-negative
    bool vegetarian = false; // whether the dish is vegetarian
};
```

```
struct Restaurant {
    string name; // Restaurant name
    string description; // Short description
    MenuItem* menus = nullptr; // Pointer to dynamically
    allocated array of menu items
    int menuSize = 0; // The size of the menu
    double averagePrice = 0.0; // The average price of the
    items
};
```

```
double minimum = 0.0; // The price of the
cheapest menu item
double maximum = 0.0; // The price of the most
expensive menu item
bool vegetarianOptions = false; // Derived: at least one
menu item is vegetarian
};
```

```
void clear_input() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
```

```
int readPositiveInt(const string &prompt) {
    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) {
            clear_input();
            if (v > 0) return v;
            cout << "Value must be a positive integer." <<
endl;
        } else {
            cout << "Invalid input." << endl;
            clear_input();
        }
    }
}
```

```
double readNonNegativeDouble(const string &prompt) {
    while (true) {
        cout << prompt;
        double v;
        if (cin >> v) {
            clear_input();
            if (v >= 0.0) return v;
            cout << "Price must be non-negative." << endl;
        } else {
            cout << "Invalid number." << endl;
            clear_input();
        }
    }
}
```

```
string readNonEmptyLine(const string &prompt) {
    while (true) {
        cout << prompt;
        string s;
        getline(cin, s);
        if (!s.empty()) return s;
        cout << "This field cannot be empty." << endl;
    }
}
```

```
bool readYesNo(const string &prompt) {
    while (true) {
        cout << prompt << " (y/n): ";
        string s; getline(cin, s);
        if (s.size()) {
            char c =
static_cast<char>(tolower(static_cast<unsigned char>(s[0])));
            if (c == 'y') return true;
            if (c == 'n') return false;
        }
        cout << "Please enter 'y' or 'n'." << endl;
    }
}
```

```

int main() {
    cout << "-- Restaurant Top Dishes --" << endl;
    int n = readPositiveInt("Enter number of restaurants: ");
    Restaurant* rs = new Restaurant[n];

    for (int i = 0; i < n; ++i) {
        cout << "\nRestaurant #" << (i + 1) << endl;
        rs[i].name = readNonEmptyLine("Name: ");
        rs[i].description = readNonEmptyLine("Short
description: ");

        rs[i].menuSize = readPositiveInt("Number of menu
items: ");
        rs[i].menus = new MenuItem[rs[i].menuSize];

        double sum = 0.0;
        rs[i].vegetarianOptions = false;
        for (int j = 0; j < rs[i].menuSize; ++j) {
            cout << "    Menu item #" << (j + 1) << endl;
            rs[i].menus[j].name = readNonEmptyLine("
Dish name: ");
            rs[i].menus[j].picture = readNonEmptyLine("
Picture link: ");
            rs[i].menus[j].price = readNonNegativeDouble("
Price: ");
            rs[i].menus[j].vegetarian = readYesNo("        Is
this dish vegetarian?");

            if (j == 0) {
                rs[i].minimum = rs[i].menus[j].price;
                rs[i].maximum = rs[i].menus[j].price;
            } else {
                if (rs[i].menus[j].price < rs[i].minimum)
                    rs[i].minimum = rs[i].menus[j].price;
                if (rs[i].menus[j].price > rs[i].maximum)
                    rs[i].maximum = rs[i].menus[j].price;
            }

            sum += rs[i].menus[j].price;
            if (rs[i].menus[j].vegetarian)
                rs[i].vegetarianOptions = true;
        }
        rs[i].averagePrice = sum /
static_cast<double>(rs[i].menuSize);
    }

    cout << "\n-- Filter by average price range --" << endl;
    double minP = readNonNegativeDouble("Minimum
average price: ");
    double maxP = readNonNegativeDouble("Maximum
average price: ");
    while (maxP < minP) {
        cout << "Maximum cannot be less than minimum.
Please re-enter." << endl;
        minP = readNonNegativeDouble("Minimum average
price: ");
        maxP = readNonNegativeDouble("Maximum average
price: ");
    }

    cout << "\nRestaurants within the range [" << minP << ", "
<< maxP << "]:" << endl;
    bool any = false;
    for (int i = 0; i < n; ++i) {
        if (rs[i].averagePrice >= minP && rs[i].averagePrice
<= maxP) {

```

```

        any = true;
        cout << "- " << rs[i].name << " | avg: " <<
rs[i].averagePrice << endl;
    }
}
if (!any) cout << "No restaurants found in the given price
range." << endl;

// clean up allocated menus
for (int i = 0; i < n; ++i) {
    delete[] rs[i].menus;
    rs[i].menus = nullptr;
}
delete[] rs;
return 0;
}

```

14. Inventory Bins

Write a program that simulates inventory bins in a warehouse. Each bin holds a number of the same type of parts. The program should use a structure that keeps the following data:

Description of the part kept in the bin

Number of parts in the bin

The program should have an array of 10 bins, initialized with the following data:

Part Description	Number of Parts in the Bin
Valve	10
Bearing	5
Bushing	15
Coupling	21
Flange	7
Gear	5
Gear Housing	5
Vacuum Gripper	25
Cable	18
Rod	12

The program should have the following functions:

AddParts—increases a specific bin's part count by a specified number.

RemoveParts—decreases a specific bin's part count by a specified number.

When the program runs, it should repeat a loop that performs the following steps: The user should see a list of what each bin holds and how many parts are in each bin. The user can choose to either quit the program or select a bin. When a bin is selected, the user can either add parts to it or remove parts from it. The loop then repeats, showing the updated bin data on the screen.

Input Validation: No bin can hold more than 30 parts, so don't let the user add more than a bin can hold. Also, don't accept negative values for the number of parts being added or removed.

```
#include <iostream>
```

```
#include <iomanip>
```

```
#include <limits>
```

```
#include <string>
```

```
using namespace std;
```

```

struct Bin {
    string description;
    int parts;
};

```

```

void clear_input() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

```

```

int readIntInRange(const string &prompt, int lo, int hi) {
    while (true) {
        cout << prompt;
        int v;
        if (cin >> v) {
            clear_input();
            if (v >= lo && v <= hi) return v;
            cout << "Value out of range." << endl;
        } else {
            cout << "Invalid input." << endl;
            clear_input();
        }
    }
}

void AddParts(Bin bins[], int index, int amount, int maxParts) {
    bins[index].parts += amount;
    if (bins[index].parts > maxParts) bins[index].parts =
maxParts;
}

void RemoveParts(Bin bins[], int index, int amount) {
    bins[index].parts -= amount;
    if (bins[index].parts < 0) bins[index].parts = 0;
}

void printBins(const Bin bins[], int n) {
    cout << "\n-- Inventory Bins --" << endl;
    cout << left << setw(4) << "No." << setw(18) << "Part" <<
"Count" << endl;
    for (int i = 0; i < n; ++i) {
        cout << left << setw(4) << (i + 1) << setw(18) <<
bins[i].description << bins[i].parts << endl;
    }
}

int main() {
    const int BIN_COUNT = 10;
    const int MAX_PARTS = 30;

    Bin bins[BIN_COUNT] = {
        {"Valve", 10},
        {"Bearing", 5},
        {"Bushing", 15},
        {"Coupling", 21},
        {"Flange", 7},
        {"Gear", 5},
        {"Gear Housing", 5},
        {"Vacuum Gripper", 25},
        {"Cable", 18},
        {"Rod", 12},
    };

    while (true) {
        printBins(bins, BIN_COUNT);
        int choice = readIntInRange("Select a bin (1-10) or 0
to quit: ", 0, BIN_COUNT);
        if (choice == 0) break;

        int idx = choice - 1;
        int op = readIntInRange("1) Add parts  2) Remove
parts: ", 1, 2);

        if (op == 1) {
            int spaceLeft = MAX_PARTS - bins[idx].parts;
            int amount = readIntInRange("Number of parts
to add: ", 0, spaceLeft);

```

```

        AddParts(bins, idx, amount, MAX_PARTS);
    } else {
        int amount = readIntInRange("Number of parts
to remove: ", 0, bins[idx].parts);
        RemoveParts(bins, idx, amount);
    }
}

return 0;
}

```


Chapter 13 challenging problem

1. Date

Design a class called Date. The class should store a date in three integers: month, day, and year. There should be member functions to print the date in the following forms:

12/25/2018

December 25, 2018

25 December 2018

Demonstrate the class by writing a complete program implementing it.

Input Validation: Do not accept values for the day greater than 31 or less than 1. Do not accept values for the month greater than 12 or less than 1.

```
#include<iostream>
```

```
#include<string>
```

```
class Date
{
    private:
        int day;
        int month;
        int year;
        std::string months[12] = {"January", "February",
"March", "April", "May", "June", "July", "August",
"September", "October", "November", "December"};
    public:
        void setDate(int d, int m, int y)
        {
            day = d;
            month = m;
            year = y;
        }
        void showAmerican()
        {
```

```
std::cout<<month<<"/"<<day<<"/"<<year<<std::endl;
        }
        void showEnglish()
        {
            std::cout<<months[month-1]<<" "<<day<<",
"<<year<<std::endl;
        }
        void showUnofficial()
        {
            std::cout<<day<<" "<<months[month-1]<<"
"<<year<<std::endl;
        }
};
```

```
int main()
{
    int d, m, y;
    Date date;
    std::cout<<"Enter day: ";
    std::cin>>d;
    while(d<1 || d>31)
    {
        std::cout<<"Invalid day. Enter again: ";
        std::cin>>d;
    }
    std::cout<<"Enter month: ";
    std::cin>>m;
    while(m<1 || m>12)
    {
        std::cout<<"Invalid month. Enter again: ";
        std::cin>>m;
    }
}
```

```
std::cout<<"Enter year: ";
std::cin>>y;
date.setDate(d, m, y);
int format;
std::cout<<"Choose date format (1: American, 2: English,
3: Unofficial): ";
std::cin>>format;
switch(format)
{
    case 1:
        date.showAmerican();
        break;
    case 2:
        date.showEnglish();
        break;
    case 3:
        date.showUnofficial();
        break;
    default:
        std::cout<<"Invalid format choice."<<std::endl;
}
return 0;
}
```

2. Employee Class

Write a class named Employee that has the following member variables:

- name—a string that holds the employee's name
 - idNumber—a n int variable that holds the employee's ID number
 - department—a string that holds the name of the department where the employee works
 - position—a string that holds the employee's job title
- The class should have the following constructors:
- A constructor that accepts the following values as arguments and assigns them to the appropriate member variables: employee's name, employee's ID number, depart ment, and position.
 - A constructor that accepts the following values as arguments and assigns them to the appropriate member variables: employee's name and ID number. The department and position fields should be assigned an empty string ("").
 - A default constructor that assigns empty strings ("") to the name, department, and position member variables, and 0 to the idNumber member variable. Write appropriate mutator functions that store values in these member variables and accessor functions that return the values in these member variables. Once you have written the class, write a separate program that creates three Employee objects to hold the following data:

Name	ID Number	Department	Position
Susan Meyers	47899	Accounting	Vice President
Mark Jones	39119	IT	Programmer
Joy Rogers	81774	Manufacturing	Engineer

The program should store this data in the three objects and then display the data for each employee on the screen.

```
#include<iostream>
```

```
#include<string>
```

```
class Employee
{
    private:
        std::string name;
        long long idNumber;
        std::string department;
        std::string position;
    public:
        //constructors
```

```

Employee(std::string n, long long id, std::string dept,
std::string pos)
{
    name = n;
    idNumber = id;
    department = dept;
    position = pos;
}

Employee(std::string n, long long id)
{
    name = n;
    idNumber = id;
    department = "";
    position = "";
}

Employee()
{
    name = "";
    idNumber = 0;
    department = "";
    position = "";
}

//accessor functions
void showInfo()
{
    std::cout<<"Name: "<<name<<std::endl;
    std::cout<<"ID Number:
"<<idNumber<<std::endl;
    std::cout<<"Department:
"<<department<<std::endl;
    std::cout<<"Position: "<<position<<std::endl;
}

};

int main()
{
    Employee emp1("Susan Meyers", 47899, "Accounting",
"Vice President");
    Employee emp2("Mark Jones", 39119, "IT",
"Programmer");
    Employee emp3("Joy Rogers", 81774, "Manufacturing",
"Engineer");

    std::cout<<"Employee 1 details:"<<std::endl;
    emp1.showInfo();
    std::cout<<std::endl;

    std::cout<<"Employee 2 details:"<<std::endl;
    emp2.showInfo();
    std::cout<<std::endl;

    std::cout<<"Employee 3 details:"<<std::endl;
    emp3.showInfo();
    std::cout<<std::endl;

    return 0;
}

```

3. Car Class

Write a class named Car that has the following member variables:

- yearModel—an int that holds the car's year model
- make—a string that holds the make of the car
- speed—an int that holds the car's current speed

In addition, the class should have the following constructor and other member functions:

- Constructor—The constructor should accept the car's year model and make as arguments. These values should be assigned to the object's yearModel and make member variables. The constructor should also assign 0 to the speed member variables.
- Accessor—appropriate accessor functions to get the values stored in an object's yearModel, make, and speed member variables
- accelerate—The accelerate function should add 5 to the speed member variable each time it is called.
- brake—The brake function should subtract 5 from the speed member variable each time it is called.

Demonstrate the class in a program that creates a Car object, then calls the accelerate function five times. After each call to the accelerate function, get the current speed of the car and display it. Then, call the brake function five times. After each call to the brake function, get the current speed of the car and display it.

```
#include<bits/stdc++.h>
```

```
#include<cstring>
```

```

class Car
{
    private:
        int yearModel;
        std::string make;
        int speed;
    public:
        Car(int ym, std::string m)
        {
            yearModel = ym;
            make = m;
            speed = 0;
        }

        void Accessor(std::string s)
        {
            if (s == "yearModel")
                std::cout << "Year Model: " << yearModel
<< std::endl;
            else if (s == "make")
                std::cout << "Make: " << make << std::endl;
            else if (s == "speed")
                std::cout << "Speed: " << speed << " mph"
<< std::endl;
        }

        void Accelerate()
        {
            speed += 5;
        }

        void Brake()
        {
            speed -= 5;
        }
};

int main()
{
    Car Benz(2020, "Mercedes");
    for(int i=0;i<5;i++)
    {
        Benz.Accelerate();
        Benz.Accessor("speed");
    }
    for(int i=0;i<5;i++)

```

```

    {
        Benz.Brake();
        Benz.Accessor("speed");
    }
    return 0;
}

```

4. Radio Stations

Write a class named BroadcastStation that has member variables for the following data:

- Name of the station
- Frequency
- Type of band (FM or AM)
- The genre of the station (country, sports, rock, classic, etc.)

The BroadcastStation class should have a constructor that accepts an argument for each member variable. The BroadcastStation class should also have accessor and mutator functions for each member variable.

Next, write a class RadioReceiver that represents a basic radio that can store two preset channels. The RadioReceiver class should have member variables for:

- The current volume of the radio
- The name, frequency, and type of the current radio channel
- Two instances of the BroadcastStation class for the two preset channels

The RadioReceiver class should have a constructor with no arguments: all member variables should be initialized to factory default settings. The RadioReceiver class should have accessor and mutator functions for each member variable. Because in a typical radio the volume and frequency are changed by turning a knob, also supply mutator functions for the volume and frequency that increase and decrease the current value by 1 unit. It should also be possible to store the current radio channel in one of the preset channels, and to change the current radio channel by selecting one of the preset channels.

Also write a function that displays the current state of the RadioReceiver, including the preset channels.

Next, demonstrate the correct working of your classes in a program that creates an instance of the RadioReceiver and then performs actions with the RadioReceiver. Make sure that all information of the RadioReceiver is displayed after each interaction.

4.cpp 文件

```

#include<iostream>
#include<cstring>
#include "BroadcastStation.h"
#include "RadioReceive.h"
int main()
{
    BroadcastStation stationA("Music", 99.5, "FM", "Pop");
    BroadcastStation stationB("News", 720.0, "AM", "News");

    RadioReceive myRadio;
    std::cout << "[1] Factory defaults" << std::endl;
    myRadio.displayCurrentState();

    std::cout << "\n[2] Set current station = stationA, set
volume = 75" << std::endl;
    myRadio.setusepreset(stationA);
    myRadio.setVolume(75.0);
    myRadio.displayCurrentState();

    std::cout << "\n[3] Store current channel into Preset 1" <<
std::endl;
    myRadio.storeCurrentToPreset1();
    myRadio.displayCurrentState();

```

```

    std::cout << "\n[4] Switch current station to stationB, then
store into Preset 2" << std::endl;
    myRadio.setusepreset(stationB);
    myRadio.storeCurrentToPreset2();
    myRadio.displayCurrentState();

    std::cout << "\n[5] Turn knobs: volume +1, frequency +1"
<< std::endl;
    myRadio.volumeUp();
    myRadio.frequencyUp();
    myRadio.displayCurrentState();

    std::cout << "\n[6] Select Preset 1" << std::endl;
    myRadio.selectPreset1();
    myRadio.displayCurrentState();

    std::cout << "\n[7] Turn knobs: volume -1, frequency -1"
<< std::endl;
    myRadio.volumeDown();
    myRadio.frequencyDown();
    myRadio.displayCurrentState();

    std::cout << "\n[8] Select Preset 2" << std::endl;
    myRadio.selectPreset2();
    myRadio.displayCurrentState();

    return 0;
}

BroadcastStation::BroadcastStation(std::string name, double
frequency, std::string type, std::string genre)
{
    Name = name;
    Frequency = frequency;
    Type = type;
    Genre = genre;
}
void BroadcastStation::setterName(const std::string name)
{
    Name = name;
}
void BroadcastStation::setterFrequency(double frequency)
{
    Frequency = frequency;
}
void BroadcastStation::setterType(const std::string type)
{
    Type = type;
}
void BroadcastStation::setterGenre(const std::string genre)
{
    Genre = genre;
}
std::string BroadcastStation::getterName() const
{
    return Name;
}
double BroadcastStation::getterFrequency() const
{
    return Frequency;
}
std::string BroadcastStation::getterType() const
{
    return Type;
}
std::string BroadcastStation::getterGenre() const
{

```

```

    return Genre;
}

void RadioReceive::frequencyUp()
{
    CurrFrequency += 1.0;
}

void RadioReceive::frequencyDown()
{
    CurrFrequency -= 1.0;
}

void RadioReceive::volumeUp()
{
    Volume += 1.0;
}

void RadioReceive::volumeDown()
{
    Volume -= 1.0;
}

void RadioReceive::presetStation(const std::string
stationName, const double stationFrequency, const std::string
stationType, const std::string stationGenre)
{
    BroadcastStation presetStation(stationName,
stationFrequency, stationType, stationGenre);
    setusepreset(presetStation);
}

void RadioReceive::setusepreset(const BroadcastStation
station)
{
    setCurrName(station.getName());
    setCurrFrequency(station.getFrequency());
    setCurrType(station.getType());
    setCurrGenre(station.getGenre());
}

void RadioReceive::storeCurrentToPreset1()
{
    Station1 = BroadcastStation(CurrName, CurrFrequency,
CurrType, CurrGenre);
}

void RadioReceive::storeCurrentToPreset2()
{
    Station2 = BroadcastStation(CurrName, CurrFrequency,
CurrType, CurrGenre);
}

void RadioReceive::selectPreset1()
{
    setusepreset(Station1);
}

void RadioReceive::selectPreset2()
{
    setusepreset(Station2);
}

void RadioReceive::displayCurrentState() const
{
    std::cout << "Current Station: " << CurrName << std::endl;
    std::cout << "Frequency: " << CurrFrequency << " MHz" <<
std::endl;
    std::cout << "Type: " << CurrType << std::endl;
    std::cout << "Genre: " << CurrGenre << std::endl;
    std::cout << "Volume: " << Volume << std::endl;

    std::cout << "Preset 1: " << Station1.getName()
        << " | " << Station1.getFrequency() << "
MHz"
        << " | " << Station1.getType()
        << " | " << Station1.getGenre()

```

```

        << std::endl;
    std::cout << "Preset 2: " << Station2.getName()
        << " | " << Station2.getFrequency() << "
MHz"
        << " | " << Station2.getType()
        << " | " << Station2.getGenre()
        << std::endl;
}

BroadcastStation.h 文件
#ifndef BROADCASTSTATION_H
#define BROADCASTSTATION_H
#include <string>
class BroadcastStation
{
    private:
        std::string Name;
        double Frequency;
        std::string Type;
        std::string Genre;
    public:
        BroadcastStation(std::string name="", double
frequency=0.0, std::string type="", std::string genre="");
        //@set info
        void setName(const std::string name);
        void setFrequency(double frequency);
        void setType(const std::string type);
        void setGenre(const std::string genre);
        //@get info
        std::string getName() const;
        double getFrequency() const;
        std::string getType() const;
        std::string getGenre() const;
};
#endif

RadioReceive.h 文件
#ifndef radio_receive_h
#define radio_receive_h
#include<cstring>
#include "BroadcastStation.h"
class RadioReceive
{
    private:
        BroadcastStation Station1, Station2;
        double Volume;
        std::string CurrName;
        double CurrFrequency;
        std::string CurrType;
        std::string CurrGenre;
    public:
        RadioReceive()
        {
            Volume = 0.0;
            CurrName = "";
            CurrFrequency = 0.0;
            CurrType = "";
            CurrGenre = "";
            Station1 = BroadcastStation("", 0.0, "", "");
            Station2 = BroadcastStation("", 0.0, "", "");
        }
        //@set info
        void setStation1(const BroadcastStation station)
        {
            Station1 = station;
        }
        void setStation2(const BroadcastStation station)
        {

```

```

        Station2 = station;
    }
    void setVolume(const double volume)
    {
        Volume = volume;
    }
    void setCurrName(const std::string name)
    {
        CurrName = name;
    }
    void setCurrFrequency(const double frequency)
    {
        CurrFrequency = frequency;
    }
    void setCurrType(const std::string type)
    {
        CurrType = type;
    }
    void setCurrGenre(const std::string genre)
    {
        CurrGenre = genre;
    }
    //@get info
    BroadcastStation getStation1() const
    {
        return Station1;
    }
    BroadcastStation getStation2() const
    {
        return Station2;
    }
    double getVolume() const
    {
        return Volume;
    }
    std::string getCurrName() const
    {
        return CurrName;
    }
    double getCurrFrequency() const
    {
        return CurrFrequency;
    }
    std::string getCurrType() const
    {
        return CurrType;
    }
    std::string getCurrGenre() const
    {
        return CurrGenre;
    }
    //@knobs (increase/decrease by 1 unit)
    void frequencyUp();
    void frequencyDown();
    void volumeUp();
    void volumeDown();

    //@presets
    void presetStation(const std::string stationName,
const double stationFrequency, const std::string stationType,
const std::string stationGenre);
    void setusepreset(const BroadcastStation station);
    void storeCurrentToPreset1();
    void storeCurrentToPreset2();
    void selectPreset1();
    void selectPreset2();
    //@display current state

```

```

    void displayCurrentState() const;
};
#endif

```

5. RetailItem Class

Write a class named RetailItem that holds data about an item in a retail store. The class should have the following member variables:

- description—a string that holds a brief description of the item
- unitsOnHand—an int that holds the number of units currently in inventory
- price—a double that holds the item's retail price

Write a constructor that accepts arguments for each member variable, appropriate mutator functions that store values in these member variables, and accessor functions that return the values in these member variables. Once you have written the class, write a separate program that creates three RetailItem objects and stores the following data in them:

	Description	Units On Hand	Price
Item #1	Jacket	12	59.95
Item #2	Designer Jeans	40	34.95
Item #3	Shirt	20	24.95

5.cpp 文件

```

#include <iostream>
#include "RetailItem.h"
#include <string>
int main()
{
    RetailItem item1("Jacket", 12, 59.95);
    RetailItem item2("Designer Jeans", 40, 34.95);
    RetailItem item3("Shirt", 20, 24.95);

    std::cout << "Item 1: " << item1.getDescription() << ",
Units on Hand: " << item1.getUnitsOnHand() << ", Price: $" <<
item1.getPrice() << std::endl;
    std::cout << "Item 2: " << item2.getDescription() << ",
Units on Hand: " << item2.getUnitsOnHand() << ", Price: $" <<
item2.getPrice() << std::endl;
    std::cout << "Item 3: " << item3.getDescription() << ",
Units on Hand: " << item3.getUnitsOnHand() << ", Price: $" <<
item3.getPrice() << std::endl;

    return 0;
}

```

RetailItem 文件

```

#ifndef RETAILITEM_H
#define RETAILITEM_H
#include <string>
class RetailItem
{
    private:
        std::string description;
        int unitsOnHand;
        double price;
    public:
        RetailItem(std::string desc, int units, double prc)
        {
            description = desc;
            unitsOnHand = units;
            price = prc;
        }
        void updateDescription(std::string desc)
        {
            description = desc;
        }
        void updateUnitsOnHand(int units)
        {

```

```

        unitsOnHand = units;
    }
    void updatePrice(double prc)
    {
        price = prc;
    }
    std::string getDescription() const
    {
        return description;
    }
    int getUnitsOnHand() const
    {
        return unitsOnHand;
    }
    double getPrice() const
    {
        return price;
    }
};
#endif

```

6. Inventory Class

Design an Inventory class that can hold information and calculate data for items in a retail store's inventory. The class should have the following private member variables:

Variable	Name Description
itemNumber	An int that holds the item's item number.
quantity	An int for holding the quantity of the items on hand.
cost	A double for holding the wholesale per-unit cost of the item
totalCost	A double for holding the total inventory cost of the item (calculated as quantity times cost).

The class should have the following *public* member functions:

Member Function Description

Default Constructor	Sets all the member variables to 0.
Constructor #2	Accepts an item's number, cost, and quantity as arguments. The function should copy these values to the appropriate member variables and then call the setTotalCost function.
setItemNumber	Accepts an integer argument that is copied to the itemNumber member variable.
setQuantity	Accepts an integer argument that is copied to the quantity member variable.
setCost	Accepts a double argument that is copied to the cost member variable.
setTotalCost	Calculates the total inventory cost for the item (quantity times cost) and stores the result in totalCost.
getItemNumber	Returns the value in itemNumber.
getQuantity	Returns the value in quantity.
getCost	Returns the value in cost.
getTotalCost	Returns the value in totalCost.

Demonstrate the class in a driver program.

Input Validation: Do not accept negative values for item number, quantity, or cost.

6.cpp 文件

```

#include<iostream>
#include "Inventory.h"
int main()
{
    Inventory item1(1001, 5, 10.0);
    Inventory item2(1002, 3, 20.0);

    std::cout << "Item 1:" << std::endl;
    std::cout << "Item Number: " << item1.getItemNumber()
<< std::endl;
    std::cout << "Quantity: " << item1.getQuantity() <<
std::endl;
    std::cout << "Cost per Item: $" << item1.getCost() <<

```

```

std::endl;
    std::cout << "Total Cost: $" << item1.getTotalCost() <<
std::endl;

    std::cout << "\nItem 2:" << std::endl;
    std::cout << "Item Number: " << item2.getItemNumber()
<< std::endl;
    std::cout << "Quantity: " << item2.getQuantity() <<
std::endl;
    std::cout << "Cost per Item: $" << item2.getCost() <<
std::endl;
    std::cout << "Total Cost: $" << item2.getTotalCost() <<
std::endl;

    // Demonstrate input validation (negative values are not
accepted)
    Inventory badItem(-1, -5, -9.99);
    std::cout << "\nBad Item (after validation):" << std::endl;
    std::cout << "Item Number: " <<
badItem.getItemNumber() << std::endl;
    std::cout << "Quantity: " << badItem.getQuantity() <<
std::endl;
    std::cout << "Cost per Item: $" << badItem.getCost() <<
std::endl;
    std::cout << "Total Cost: $" << badItem.getTotalCost() <<
std::endl;

    return 0;
}

```

Inventory.h 文件

```

#ifndef INVENTORY_H
#define INVENTORY_H
#include <string>
class Inventory
{
    private:
        int itemNumber;
        int quantity;
        double cost;
        double totalCost;
    public:
        Inventory()
        {
            itemNumber = 0;
            quantity = 0;
            cost = 0.0;
            totalCost = 0.0;
        }

        Inventory(int itemNum, int qty, double cst)
        {
            setItemNumber(itemNum);
            setQuantity(qty);
            setCost(cst);
            setTotalCost();
        }

        // Mutators (Input Validation: do not accept negative
values)
        void setItemNumber(int itemNum)
        {
            itemNumber = (itemNum < 0) ? 0 : itemNum;
        }
        void setQuantity(int qty)
        {
            quantity = (qty < 0) ? 0 : qty;
            setTotalCost();
        }

```

```

    }
    void setCost(double cst)
    {
        cost = (cst < 0.0) ? 0.0 : cst;
        setTotalCost();
    }
    void setTotalCost()
    {
        totalCost = quantity * cost;
    }

    // Accessors
    int getItemNumber() const
    {
        return itemNumber;
    }
    int getQuantity() const
    {
        return quantity;
    }
    double getCost() const
    {
        return cost;
    }
    double getTotalCost() const
    {
        return totalCost;
    }
};
#endif

```

7. Length Class

Design a Length class that has two member variables: meters and centimeters. The class should have a constructor, an accessor, and mutator functions for the member variables and a member function getLength() that returns the length in centimeters. Demonstrate the class by writing a separate program that creates two instances of the class. The program should ask the user to enter the lengths in meters and centimeters, which are stored in the Length objects. The program should then compare the lengths of the two objects with the help of the getLength member function.

```
#include <iostream>
```

```

class Length
{
private:
    int meters;
    int centimeters;

public:
    Length(int m = 0, int cm = 0)
    {
        meters = m;
        centimeters = cm;
    }

    void setMeters(int m)
    {
        meters = m;
    }

    void setCentimeters(int cm)
    {
        centimeters = cm;
    }

    int getMeters() const

```

```

    {
        return meters;
    }

    int getCentimeters() const
    {
        return centimeters;
    }

    int getLength() const
    {
        return meters * 100 + centimeters;
    }
};

int main()
{
    Length a, b;

    int m = 0;
    int cm = 0;

    std::cout << "Enter length 1 (meters then centimeters): ";
    std::cin >> m >> cm;
    a.setMeters(m);
    a.setCentimeters(cm);

    std::cout << "Enter length 2 (meters then centimeters): ";
    std::cin >> m >> cm;
    b.setMeters(m);
    b.setCentimeters(cm);

    if (a.getLength() > b.getLength())
    {
        std::cout << "Length 1 is longer." << std::endl;
    }
    else if (a.getLength() < b.getLength())
    {
        std::cout << "Length 2 is longer." << std::endl;
    }
    else
    {
        std::cout << "The two lengths are equal." <<
std::endl;
    }

    return 0;
}

```

8. Circle Class

Write a Circle class that has the following member variables:

- radius—a double
 - pi—a double initialized with the value 3.14159
- The class should have the following member functions:
- Default Constructor—a default constructor that sets radius to 0.0
 - Constructor—accepts the radius of the circle as an argument
 - setRadius—a mutator function for the radius variable
 - getRadius—an accessor function for the radius variable
 - getArea—returns the area of the circle, which is calculated as $\text{area} = \pi * \text{radius} * \text{radius}$
 - getDiameter—returns the diameter of the circle, which is calculated as $\text{diameter} = \text{radius} * 2$
 - getCircumference—returns the circumference of the circle, which is calculated as $\text{circumference} = 2 * \pi * \text{radius}$
- Write a program that demonstrates the Circle class by asking the user for the circle's radius, creating a Circle object, then reporting the circle's area, diameter, and circumference.


```
#include <iostream>
```

```
class Circle
```

```
{
private:
    double radius;
    double pi;

public:
    Circle()
    {
        radius = 0.0;
        pi = 3.14159;
    }

    Circle(double r)
    {
        radius = r;
        pi = 3.14159;
    }

    void setRadius(double r)
    {
        radius = r;
    }

    double getRadius() const
    {
        return radius;
    }

    double getArea() const
    {
        return pi * radius * radius;
    }

    double getDiameter() const
    {
        return radius * 2;
    }

    double getCircumference() const
    {
        return 2 * pi * radius;
    }
};

int main()
{
    double r = 0.0;
    std::cout << "Enter the circle's radius: ";
    std::cin >> r;

    Circle c(r);
    std::cout << "Area: " << c.getArea() << std::endl;
    std::cout << "Diameter: " << c.getDiameter() << std::endl;
    std::cout << "Circumference: " << c.getCircumference() <<
    std::endl;

    return 0;
}
```

9. Number of Visitors

All events have a certain number of visitors permitted. When the maximum number of visitors is reached, nobody can enter unless somebody leaves the premises. For this purpose, most events implement counter systems that increment the number of people every time somebody enters

the event, and decrement the number of people when some body leaves the event. Typically, the entrance is blocked when the event is fully occupied.

Design a VisitorCounter class that stores both the current and the maximum capacity of the event. The constructor for the class should have one argument: the maximum capacity of the function. For both fields, a member accessor function should be implemented. For the mutator, only functions to add a visitor and to remove a visitor should be implemented. The mutator to add a visitor should return true as long as the maximum capacity is not reached. It should return false when the visitor is not allowed to enter because the maximum capacity has been reached.

Write a program that demonstrates your VisitorCounter class.

```
#include <iostream>
```

```
class VisitorCounter
```

```
{
private:
    int current;
    int maximum;

public:
    VisitorCounter(int maxCapacity)
    {
        maximum = maxCapacity;
        current = 0;
    }

    int getCurrent() const
    {
        return current;
    }

    int getMaximum() const
    {
        return maximum;
    }

    bool addVisitor()
    {
        if (current < maximum)
        {
            ++current;
            return true;
        }
        return false;
    }

    void removeVisitor()
    {
        if (current > 0)
        {
            --current;
        }
    }
};
```

```
int main()
```

```
{
    VisitorCounter counter(5);

    std::cout << "Maximum capacity: " <<
    counter.getMaximum() << std::endl;
    std::cout << "Current visitors: " << counter.getCurrent() <<
    std::endl;
```

```

    for (int i = 0; i < 6; ++i)
    {
        bool allowed = counter.addVisitor();
        std::cout << "Add visitor: " << (allowed ? "allowed" :
"blocked")
                << ", current = " << counter.getCurrent()
<< std::endl;
    }

    counter.removeVisitor();
    std::cout << "Remove visitor, current = " <<
counter.getCurrent() << std::endl;

    bool allowed = counter.addVisitor();
    std::cout << "Add visitor: " << (allowed ? "allowed" :
"blocked")
            << ", current = " << counter.getCurrent() <<
std::endl;

    return 0;
}

```

10. Number Array Class

Design a class that has an array of floating-point numbers. The constructor should accept an integer argument and dynamically allocate the array to hold that many numbers. The destructor should free the memory held by the array. In addition, there should be member functions to perform the following operations:

- Store a number in any element of the array
- Retrieve a number from any element of the array
- Return the highest value stored in the array
- Return the lowest value stored in the array
- Return the average of all the numbers stored in the array

Demonstrate the class in a program.

```
#include <iostream>
```

```

class NumberArray
{
private:
    double *arr;
    int size;

public:
    NumberArray(int n)
    {
        size = n;
        arr = new double[size];
        for (int i = 0; i < size; ++i)
        {
            arr[i] = 0.0;
        }
    }

    ~NumberArray()
    {
        delete[] arr;
    }

    void set(int index, double value)
    {
        arr[index] = value;
    }

    double get(int index) const
    {
        return arr[index];
    }
}

```

```

double highest() const
{
    double hi = arr[0];
    for (int i = 1; i < size; ++i)
    {
        if (arr[i] > hi)
        {
            hi = arr[i];
        }
    }
    return hi;
}

```

```

double lowest() const
{
    double lo = arr[0];
    for (int i = 1; i < size; ++i)
    {
        if (arr[i] < lo)
        {
            lo = arr[i];
        }
    }
    return lo;
}

```

```

double average() const
{
    double sum = 0.0;
    for (int i = 0; i < size; ++i)
    {
        sum += arr[i];
    }
    return sum / size;
}

```

```
};
```

```

int main()
{
    int n = 0;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    NumberArray numbers(n);

    for (int i = 0; i < n; ++i)
    {
        double value = 0.0;
        std::cout << "Enter value #" << (i + 1) << ": ";
        std::cin >> value;
        numbers.set(i, value);
    }

    std::cout << "Highest: " << numbers.highest() << std::endl;
    std::cout << "Lowest: " << numbers.lowest() << std::endl;
    std::cout << "Average: " << numbers.average() <<
std::endl;

    return 0;
}

```

11. Array of Ball Objects

Design a Ball class that has member variables for the radius and color of a ball. The class should have a member function getVolume that returns the volume of the ball. The volume of a sphere with radius r can be calculated using the following formula:

$$Volume = \frac{4}{3}\pi r^3$$

Create an array of five Ball objects. The program should ask the user for the radius and color of each ball object and then display the ball with the maximum volume.

```
#include <iostream>
#include <string>
#include <cmath>
#include <vector>
using namespace std;
```

```
class Ball
{
private:
    double radius;
    string color;

public:
    Ball() : radius(0.0), color("") {}

    void setProperties(double r, string c)
    {
        radius = r;
        color = c;
    }

    double getRadius() const
    {
        return radius;
    }

    string getColor() const
    {
        return color;
    }

    double getVolume() const
    {
        const double PI = 3.14159265358979323846;
        return (4.0 / 3.0) * PI * pow(radius, 3);
    }
};
```

```
int main()
{
    const int ARRAY_SIZE = 5;
    Ball balls[ARRAY_SIZE];

    for (int i = 0; i < ARRAY_SIZE; i++)
    {
        double r;
        string c;
        cout << "number" << (i + 1) << " radius: ";
        cin >> r;
        cout << "num " << (i + 1) << " color: ";
        cin >> c;

        balls[i].setProperties(r, c);
    }

    int maxIndex = 0;
    double maxVolume = balls[0].getVolume();

    for (int i = 1; i < ARRAY_SIZE; i++)
    {
        if (balls[i].getVolume() > maxVolume)
```

```
    {
        maxVolume = balls[i].getVolume();
        maxIndex = i;
    }

    cout << "color: " << balls[maxIndex].getColor() << endl;
    cout << "radius: " << balls[maxIndex].getRadius() << endl;
    cout << "volume: " << maxVolume << endl;
    return 0;
}
```

12. Draw a Card Simulator

Write a CardDeck class that should have the following member variables:

- allCards: a vector of strings containing all the cards that can be drawn from the deck
- remainingCards: a vector of strings containing all the cards that have not yet been drawn from the deck

The CardDeck class should have the following member functions:

- A default constructor that accepts a vector of strings representing all the cards in the deck. initializes both vectors to have that contents.
- A member function named drawCard() that returns a card from the deck. It generates a random index in the remainingCards vector and that returns the card at that index. It should also remove the drawn card from the remainingCards vector. If the vector becomes empty, the deck must be restarted.
- A void member function named deal() that restarts the deck and initializes the remainingCards vector.
- A member function named getRemainingCards() that returns the number of remaining cards in the deck.

Write a program that demonstrates the CardDeck class. The program should create an instance of the class and draw all cards in the deck. Check if all cards are drawn and whether they appear in a random order.

```
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>
```

```
class CardDeck
{
private:
    std::vector<std::string> allCards;
    std::vector<std::string> remainingCards;

public:
    CardDeck(const std::vector<std::string> &cards) :
    allCards(cards), remainingCards(cards)
    {
        std::srand(static_cast<unsigned int>(std::time(0)));
    }

    void deal()
    {
        remainingCards = allCards;
    }

    std::string drawCard()
    {
        if (remainingCards.empty())
        {
            deal();
        }
    }
```

```

int randomIndex = std::rand() % remainingCards.size();

std::string drawnCard = remainingCards[randomIndex];
remainingCards.erase(remainingCards.begin() +
randomIndex);

if (remainingCards.empty())
{
    deal();
}

return drawnCard;
}

int getRemainingCards() const
{
    return remainingCards.size();
}
};

int main()
{
    std::vector<std::string> cards;
    cards.push_back("Ace of Spades");
    cards.push_back("Jack of Hearts");
    cards.push_back("Queen of Diamonds");
    cards.push_back("King of Clubs");
    cards.push_back("Ten of Spades");

    CardDeck deck(cards);

    for (int i = 0; i < 10; i++)
    {
        std::cout << "Drawn: " << deck.drawCard()
                    << " | Remaining: " <<
deck.getRemainingCards() << std::endl;
    }

    return 0;
}

```

13. Guess the Sequence

For this assignment, you will create a game using the CardDeck class from Programming Challenge 12 (Draw a Card Simulator). The program should have an instance of the CardDeck class where the cards can be alphabetically sorted.

When the game begins, the class draws seven random cards from the deck and displays the first one. The user must then guess whether the next card is lower or higher than the current card by entering "Lower" or "Higher". When the user guesses correctly, the next card is shown and the game continues.

The player wins when he guesses correctly for all cards. When he makes a wrong estimate, the player should be given the option to restart the current game, start a new game, or to quit.

```

#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>
#include <algorithm>

```

```

class CardDeck
{
private:
    std::vector<std::string> allCards;

```

```

std::vector<std::string> remainingCards;

public:
    CardDeck(const std::vector<std::string> &cards) :
allCards(cards), remainingCards(cards)
    {
        std::srand(static_cast<unsigned int>(std::time(0)));
    }

    void deal()
    {
        remainingCards = allCards;
    }

    std::string drawCard()
    {
        if (remainingCards.empty())
        {
            deal();
        }
        int randomIndex = std::rand() % remainingCards.size();
        std::string drawnCard = remainingCards[randomIndex];
        remainingCards.erase(remainingCards.begin() +
randomIndex);
        return drawnCard;
    }

    int getRemainingCards() const
    {
        return remainingCards.size();
    }
};

int main()
{
    std::vector<std::string> initialCards;
    initialCards.push_back("Ace of Spades");
    initialCards.push_back("Eight of Hearts");
    initialCards.push_back("Jack of Hearts");
    initialCards.push_back("King of Clubs");
    initialCards.push_back("Queen of Diamonds");
    initialCards.push_back("Ten of Spades");
    initialCards.push_back("Two of Clubs");

    std::sort(initialCards.begin(), initialCards.end());

    bool appRunning = true;
    while (appRunning)
    {
        CardDeck deck(initialCards);
        std::vector<std::string> gameSequence;

        for (int i = 0; i < 7; i++)
        {
            gameSequence.push_back(deck.drawCard());
        }

        bool gameOver = false;
        int currentIdx = 0;

        while (currentIdx < 6 && !gameOver)
        {
            std::cout << "\nCurrent card: " <<
gameSequence[currentIdx] << std::endl;
            std::cout << "Predict: Will the next card be 'Lower' or
'Higher'? ";
            std::string userGuess;

```

```

std::cin >> userGuess;

std::string nextCard = gameSequence[currentIdx + 1];

bool isActuallyHigher = nextCard >
gameSequence[currentIdx];
bool userWasRight = (userGuess == "Higher" &&
isActuallyHigher) ||
                    (userGuess == "Lower"
&& !isActuallyHigher);

if (userWasRight)
{
    std::cout << "Correct! The next card is: " << nextCard
<< std::endl;
    currentIdx++;
    if (currentIdx == 6)
    {
        std::cout << "Congratulations! You won by guessing
the entire sequence!" << std::endl;
    }
}
else
{
    std::cout << "Wrong! The next card was: " <<
nextCard << std::endl;
    gameOver = true;
}
}

std::cout << "\nMenu: (1) Restart current sequence (2)
New game (3) Quit: ";
int choice;
std::cin >> choice;

if (choice == 3)
{
    appRunning = false;
}
else if (choice == 2)
{
    std::cout << "Starting new game...\n";
}
else
{
    std::cout << "Restarting current sequence...\n";
}
}

return 0;
}

```

14. Fishing Game Simulation

For this assignment, you will write a program that simulates a fishing game. In this game, a six-sided die is rolled to determine what the user has caught. Each possible item is worth a certain number of fishing points. The points will not be displayed until the user has finished fishing, then a message is displayed congratulating the user depending on the number of fishing points gained.

Here are some suggestions for the game's design:

- Each round of the game is performed as an iteration of a loop that repeats as long as the player wants to fish for more items.
- At the beginning of each round, the program will ask the user whether he or she wants to continue fishing.
- The program simulates the rolling of a six-sided die (use the Die class that was demonstrated in this chapter).

- Each item that can be caught is represented by a number generated from the die. For example, 1 for "a huge fish," 2 for "an old shoe," 3 for "a little fish," and so on.
- Each item the user catches is worth a different amount of points.
- The loop keeps a running total of the user's fishing points.
- After the loop has finished, the total number of fishing points is displayed, along with a message that varies depending on the number of points earned.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
class Die
```

```
{
```

```
public:
```

```
    Die()
```

```
{
```

```
        std::srand(static_cast<unsigned int>(std::time(0)));
```

```
}
```

```
    int roll()
```

```
{
```

```
        return (std::rand() % 6) + 1;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    Die fishingDie;
```

```
    int totalPoints = 0;
```

```
    char choice;
```

```
    bool keepFishing = true;
```

```
    std::cout << "--- Welcome to the Fishing Game Simulation ---"
<< std::endl;
```

```
    while (keepFishing)
```

```
{
```

```
        std::cout << "\nDo you want to fish for an item? (y/n): ";
```

```
        std::cin >> choice;
```

```
        if (choice == 'y' || choice == 'Y')
```

```
{
```

```
            int dieValue = fishingDie.roll();
```

```
            if (dieValue == 1)
```

```
{
```

```
                std::cout << "You caught a huge fish!" << std::endl;
```

```
                totalPoints += 10;
```

```
}
```

```
            else if (dieValue == 2)
```

```
{
```

```
                std::cout << "You caught an old shoe..." << std::endl;
```

```
                totalPoints += 1;
```

```
}
```

```
            else if (dieValue == 3)
```

```
{
```

```
                std::cout << "You caught a little fish!" << std::endl;
```

```
                totalPoints += 3;
```

```
}
```

```
            else if (dieValue == 4)
```

```
{
```

```
                std::cout << "You caught a rusty can." << std::endl;
```

```
                totalPoints += 2;
```

```
}
```

```

        else if (dieValue == 5)
        {
            std::cout << "You caught a rare golden fish!" <<
std::endl;
            totalPoints += 20;
        }
        else
        {
            std::cout << "You caught a medium-sized fish!" <<
std::endl;
            totalPoints += 5;
        }
    }
    else
    {
        keepFishing = false;
    }
}

std::cout << "\nGame Over!" << std::endl;
std::cout << "Total Fishing Points: " << totalPoints <<
std::endl;

if (totalPoints >= 50)
{
    std::cout << "Incredible! You are a Master Fisherman!" <<
std::endl;
}
else if (totalPoints >= 20)
{
    std::cout << "Great job! That's a lot of fish!" << std::endl;
}
else if (totalPoints > 0)
{
    std::cout << "Not bad, but maybe you'll have better luck
next time." << std::endl;
}
else
{
    std::cout << "You didn't catch anything today." <<
std::endl;
}

return 0;
}

```

15. Mortgage Payment

Design a class that will determine the monthly payment on a home mortgage. The monthly payment with interest compounded monthly can be calculated as follows:

$$\text{Payment} = \frac{\text{Loan} \times \frac{\text{Rate}}{12} \times \text{Term}}{\text{Term} - 1}$$

where

$$\text{Term} = \left(1 + \frac{\text{Rate}}{12}\right)^{12 \times \text{Years}}$$

Payment = the monthly payment

Loan = the dollar amount of the loan

Rate = the annual interest rate

Years = the number of years of the loan

The class should have member functions for setting the loan amount, interest rate, and number of years of the loan. It should also have member functions for returning the monthly payment amount and the total amount paid to the bank at the end of the loan period. Implement the class in a complete program. Input Validation: Do not accept negative numbers for

any of the loan values.

```
#include <iostream>
```

```
#include <cmath>
```

```
#include <iomanip>
```

```
class MortgagePayment
```

```
{
```

```
private:
```

```
    double loanAmount;
```

```
    double annualRate;
```

```
    int years;
```

```
    double calculateTerm() const
```

```
{
```

```
    if (annualRate == 0 && years == 0)
```

```
        return 0;
```

```
    return std::pow((1 + annualRate / 12), 12 * years);
```

```
}
```

```
public:
```

```
    MortgagePayment() : loanAmount(0), annualRate(0),
years(0) {}
```

```
    void setLoanAmount(double loan)
```

```
{
```

```
    if (loan >= 0)
```

```
        loanAmount = loan;
```

```
}
```

```
    void setAnnualRate(double rate)
```

```
{
```

```
    if (rate >= 0)
```

```
        annualRate = rate / 100.0;
```

```
}
```

```
    void setYears(int yrs)
```

```
{
```

```
    if (yrs >= 0)
```

```
        years = yrs;
```

```
}
```

```
    double getMonthlyPayment() const
```

```
{
```

```
        double term = calculateTerm();
```

```
        if (term <= 1)
```

```
            return 0;
```

```
        return (loanAmount * (annualRate / 12) * term) / (term -
1);
    }
```

```
    double getTotalPaid() const
```

```
{
```

```
        return getMonthlyPayment() * years * 12;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    MortgagePayment mortgage;
```

```
    double loan, rate;
```

```
    int yrs;
```

```
    std::cout << "Enter loan amount: ";
```

```
    std::cin >> loan;
```

```
    std::cout << "Enter annual interest rate (e.g., 5 for 5%): ";
```

```
    std::cin >> rate;
```

```
    std::cout << "Enter number of years: ";
```

```

std::cin >> yrs;

if (loan < 0 || rate < 0 || yrs < 0)
{
    std::cout << "Invalid input. Negative values are not
accepted." << std::endl;
}
else
{
    mortgage.setLoanAmount(loan);
    mortgage.setAnnualRate(rate);
    mortgage.setYears(yrs);

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "\nMonthly Payment: $" <<
mortgage.getMonthlyPayment() << std::endl;
    std::cout << "Total Amount Paid: $" <<
mortgage.getTotalPaid() << std::endl;
}

return 0;
}

```

16. Freezing and Boiling Points

The following table lists the freezing and boiling points of several substances.

Substance	Freezing Point	Boiling Point
Ethyl alcohol	-173	172
Oxygen	-362	-306
Water	32	212

Design a class that stores a temperature in a temperature member variable and has the appropriate accessor and mutator functions. In addition to appropriate constructors, the class should have the following member functions:

- **isEthylFreezing**—This function should return the bool value true if the temperature stored in the temperature field is at or below the freezing point of ethyl alcohol. Otherwise, the function should return false.
- **isEthylBoiling**—This function should return the bool value true if the temperature stored in the temperature field is at or above the boiling point of ethyl alcohol. Otherwise, the function should return false.
- **isOxygenFreezing**—This function should return the bool value true if the temperature stored in the temperature field is at or below the freezing point of oxygen. Otherwise, the function should return false.
- **isOxygenBoiling**—This function should return the bool value true if the temperature stored in the temperature field is at or above the boiling point of oxygen. Otherwise, the function should return false.
- **isWaterFreezing**—This function should return the bool value true if the temperature stored in the temperature field is at or below the freezing point of water. Otherwise, the function should return false.
- **isWaterBoiling**—This function should return the bool value true if the temperature stored in the temperature field is at or above the boiling point of water. Otherwise, the function should return false.

Write a program that demonstrates the class. The program should ask the user to enter a temperature, then display a list of the substances that will freeze at that temperature, and those that will boil at that temperature. For example, if the temperature is -20 the class should report that water will freeze and oxygen will boil at that temperature.

```
#include <iostream>
```

```

class FreezingBoilingPoints {
private:
    int temperature;

```

```

public:
    FreezingBoilingPoints(int temp) : temperature(temp) {}

    bool isEthylFreezing() const { return temperature <= -
173; }
    bool isEthylBoiling() const { return temperature >= 172; }
    bool isOxygenFreezing() const { return temperature <= -
362; }
    bool isOxygenBoiling() const { return temperature >= -
306; }
    bool isWaterFreezing() const { return temperature <= 32; }
    bool isWaterBoiling() const { return temperature >= 212; }
};

```

```

int main() {
    int temp;
    std::cout << "Enter a temperature: ";
    std::cin >> temp;

```

```

    FreezingBoilingPoints substances(temp);

```

```

    std::cout << "At " << temp << " degrees, the substances
are in these states:" << std::endl;
    std::cout << "Ethyl Alcohol: " <<
(substances.isEthylFreezing() ? "Frozen" :
substances.isEthylBoiling() ? "Boiling" : "Liquid") << std::endl;
    std::cout << "Oxygen: " <<
(substances.isOxygenFreezing() ? "Frozen" :
substances.isOxygenBoiling() ? "Boiling" : "Gas") << std::endl;
    std::cout << "Water: " << (substances.isWaterFreezing() ?
"Frozen" : substances.isWaterBoiling() ? "Boiling" : "Liquid") <<
std::endl;

```

```

    return 0;
}

```

17. Time Class

Design a class Time that represents time in 12-hour format. The class should store time in four variables:

- hours
- minutes
- seconds
- am_pm that determines whether it is am or pm

The class should have member variables to input time in 12-hour format. It should display time in the following two formats:

1. 12-hour format (e.g., 8:15:33 pm)
2. 24-hour format (e.g., 20:15:33)

Demonstrate the class in a complete program.

Input Validation: Accept hours only within the range of 0 to 12. Accept minutes and seconds only within the range of 0 to 59. Accept only the values "am" or "pm" for am_pm. If the hour is 12, the user should not be asked to input the value of am_pm; instead am_pm should be automatically assigned "pm".

```

#include <iostream>
#include <string>
#include <iomanip>

```

```

class Time
{
private:
    int hours;
    int minutes;
    int seconds;
    std::string am_pm;

```

```

public:

```



```

Time(int h, int m, int s, std::string ap)
{
    if (h >= 0 && h <= 12 && m >= 0 && m <= 59 && s >= 0 &&
s <= 59)
    {
        hours = h;
        minutes = m;
        seconds = s;
        if (h == 12)
        {
            am_pm = "pm";
        }
        else
        {
            am_pm = ap;
        }
    }
    else
    {
        hours = 0;
        minutes = 0;
        seconds = 0;
        am_pm = "am";
    }
}

void display12Hour() const
{
    std::cout << hours << ":"
        << std::setfill('0') << std::setw(2) << minutes
<< ":"
        << std::setfill('0') << std::setw(2) << seconds
<< " "
        << am_pm << std::endl;
}

void display24Hour() const
{
    int h24 = hours;
    if (am_pm == "pm" && hours != 12)
    {
        h24 += 12;
    }
    else if (am_pm == "am" && hours == 12)
    {
        h24 = 0;
    }

    std::cout << std::setfill('0') << std::setw(2) << h24 << ":"
        << std::setfill('0') << std::setw(2) << minutes
<< ":"
        << std::setfill('0') << std::setw(2) << seconds
<< std::endl;
}

int main()
{
    int h, m, s;
    std::string ap = "am";

    std::cout << "Enter hours (0-12): ";
    std::cin >> h;
    std::cout << "Enter minutes (0-59): ";
    std::cin >> m;
    std::cout << "Enter seconds (0-59): ";
    std::cin >> s;

```

```

if (h != 12)
{
    std::cout << "Enter am or pm: ";
    std::cin >> ap;
}

Time t(h, m, s, ap);

std::cout << "\n12-hour format: ";
t.display12Hour();
std::cout << "24-hour format: ";
t.display24Hour();

return 0;
}

```

18. A Game of 21

For this assignment, you will write a program that lets the user play against the computer in a variation of the popular blackjack card game. In this variation of the game, two six-sided dice are used instead of cards. The dice are rolled, and the player tries to beat the computer's hidden total without going over 21.

Here are some suggestions for the game's design:

- Each round of the game is performed as an iteration of a loop that repeats as long as the player agrees to roll the dice, and the player's total does not exceed 21.
- At the beginning of each round, the program will ask the users whether they want to roll the dice to accumulate points.
- During each round, the program simulates the rolling of two six-sided dice. It rolls the dice first for the computer, then it asks the user if he or she wants to roll. (Use the Die class demonstrated in this chapter to simulate the dice).
- The loop keeps a running total of both the computer and the user's points.
- The computer's total should remain hidden until the loop has finished.
- After the loop has finished, the computer's total is revealed, and the player with the most points without going over 21 wins.

```

#include <iostream>
#include <cstdlib> // for rand and srand
#include <ctime>   // for time

```

```

class Die {
public:
    int roll() {
        return rand() % 6 + 1; // returns a number between
1 and 6
    }
};

```

```

void playGameOf21() {
    Die die;
    int playerTotal = 0, computerTotal = 0;
    char choice;

    srand(static_cast<unsigned int>(time(nullptr))); // Seed
for random number generation

    // Computer's turn (hidden)
    while (computerTotal < 17) { // Simple AI: keep rolling
until the total is 17 or more
        computerTotal += die.roll() + die.roll();
    }
}

```

```

// Player's turn
do {
    std::cout << "Your current total is " << playerTotal <<
". Roll dice? (y/n): ";
    std::cin >> choice;

    if (choice == 'y' || choice == 'Y') {
        playerTotal += die.roll() + die.roll();
        std::cout << "Your new total is " << playerTotal
<< std::endl;
    }

    if (playerTotal > 21) {
        std::cout << "You have gone over 21. You lose."
<< std::endl;
        return;
    }

} while (choice == 'y' || choice == 'Y');

std::cout << "The computer's total was " <<
computerTotal << std::endl;

if (playerTotal > computerTotal || computerTotal > 21) {
    std::cout << "You win!" << std::endl;
} else {
    std::cout << "You lose!" << std::endl;
}

}

int main() {
    playGameOf21();
    return 0;
}

```

19. Trivia Game

In this programming challenge, you will create a simple trivia game for two players. The program will work like this:

- Starting with player 1, each player gets a turn at answering five trivia questions. (There are a total of 10 questions.) When a question is displayed, four possible answers are also displayed. Only one of the answers is correct, and if the player selects the correct answer, he or she earns a point.
- After answers have been selected for all of the questions, the program displays the number of points earned by each player and declares the player with the highest number of points the winner.

In this program, you will design a Question class to hold the data for a trivia question. The Question class should have member variables for the following data:

- A trivia question
- Possible answer #1
- Possible answer #2
- Possible answer #3
- Possible answer #4
- The number of the correct answer (1, 2, 3, or 4)

The Question class should have appropriate constructor(s), accessor, and mutator functions.

The program should create an array of 10 Question objects, one for each trivia question. Make up your own trivia questions on the subject or subjects of your choice for the objects.

```

#include <iostream>
#include <string>

```

```

class Question
{

```

```

private:
    std::string questionText;
    std::string ans1;
    std::string ans2;
    std::string ans3;
    std::string ans4;
    int correctIndex;

public:
    Question()
    {
        questionText = "";
        ans1 = "";
        ans2 = "";
        ans3 = "";
        ans4 = "";
        correctIndex = 0;
    }

    void setQuestionText(std::string q) { questionText = q; }
    void setAns1(std::string a1) { ans1 = a1; }
    void setAns2(std::string a2) { ans2 = a2; }
    void setAns3(std::string a3) { ans3 = a3; }
    void setAns4(std::string a4) { ans4 = a4; }
    void setCorrectIndex(int c) { correctIndex = c; }

    std::string getQuestionText() const { return questionText; }
    std::string getAns1() const { return ans1; }
    std::string getAns2() const { return ans2; }
    std::string getAns3() const { return ans3; }
    std::string getAns4() const { return ans4; }
    int getCorrectIndex() const { return correctIndex; }
};

int main()
{
    Question questions[10];

    questions[0].setQuestionText("What is 2 + 2?");
    questions[0].setAns1("3");
    questions[0].setAns2("4");
    questions[0].setAns3("5");
    questions[0].setAns4("6");
    questions[0].setCorrectIndex(2);

    questions[1].setQuestionText("Which planet is known as the
Red Planet?");
    questions[1].setAns1("Earth");
    questions[1].setAns2("Mars");
    questions[1].setAns3("Jupiter");
    questions[1].setAns4("Saturn");
    questions[1].setCorrectIndex(2);

    questions[2].setQuestionText("What is the capital of
Japan?");
    questions[2].setAns1("Seoul");
    questions[2].setAns2("Beijing");
    questions[2].setAns3("Tokyo");
    questions[2].setAns4("Bangkok");
    questions[2].setCorrectIndex(3);

    questions[3].setQuestionText("Which element has the
symbol 'O'?");
    questions[3].setAns1("Gold");
    questions[3].setAns2("Silver");
    questions[3].setAns3("Oxygen");
    questions[3].setAns4("Iron");

```

```

questions[3].setCorrectIndex(3);

questions[4].setQuestionText("How many continents are
there?");
questions[4].setAns1("5");
questions[4].setAns2("6");
questions[4].setAns3("7");
questions[4].setAns4("8");
questions[4].setCorrectIndex(3);

questions[5].setQuestionText("What is the largest
mammal?");
questions[5].setAns1("Elephant");
questions[5].setAns2("Blue Whale");
questions[5].setAns3("Shark");
questions[5].setAns4("Giraffe");
questions[5].setCorrectIndex(2);

questions[6].setQuestionText("Which language is used for
C++?");
questions[6].setAns1("English");
questions[6].setAns2("Spanish");
questions[6].setAns3("C++");
questions[6].setAns4("French");
questions[6].setCorrectIndex(3);

questions[7].setQuestionText("Who painted the Mona
Lisa?");
questions[7].setAns1("Picasso");
questions[7].setAns2("Da Vinci");
questions[7].setAns3("Van Gogh");
questions[7].setAns4("Dali");
questions[7].setCorrectIndex(2);

questions[8].setQuestionText("What is the square root of
64?");
questions[8].setAns1("6");
questions[8].setAns2("7");
questions[8].setAns3("8");
questions[8].setAns4("9");
questions[8].setCorrectIndex(3);

questions[9].setQuestionText("Which ocean is the largest?");
questions[9].setAns1("Atlantic");
questions[9].setAns2("Indian");
questions[9].setAns3("Pacific");
questions[9].setAns4("Arctic");
questions[9].setCorrectIndex(3);

int p1Score = 0, p2Score = 0, choice;

std::cout << "Player 1's turn:" << std::endl;
for (int i = 0; i < 5; ++i)
{
    std::cout << "\nQuestion " << (i + 1) << ": " <<
questions[i].getQuestionText() << std::endl;
    std::cout << "1. " << questions[i].getAns1() << " 2. " <<
questions[i].getAns2()
        << " 3. " << questions[i].getAns3() << " 4. " <<
questions[i].getAns4() << std::endl;
    std::cin >> choice;
    if (choice == questions[i].getCorrectIndex())
        p1Score++;
}

std::cout << "\nPlayer 2's turn:" << std::endl;
for (int i = 5; i < 10; ++i)

```

```

{
    std::cout << "\nQuestion " << (i + 1) << ": " <<
questions[i].getQuestionText() << std::endl;
    std::cout << "1. " << questions[i].getAns1() << " 2. " <<
questions[i].getAns2()
        << " 3. " << questions[i].getAns3() << " 4. " <<
questions[i].getAns4() << std::endl;
    std::cin >> choice;
    if (choice == questions[i].getCorrectIndex())
        p2Score++;
}

std::cout << "\nPlayer 1 Score: " << p1Score << std::endl;
std::cout << "Player 2 Score: " << p2Score << std::endl;

if (p1Score > p2Score)
    std::cout << "Player 1 Wins!" << std::endl;
else if (p2Score > p1Score)
    std::cout << "Player 2 Wins!" << std::endl;
else
    std::cout << "It's a tie!" << std::endl;

return 0;
}

```

20. Patient Fees

You are to write a program that computes a patient's bill for a hospital stay. The different components of the program are

- The PatientAccount class
- The Surgery class
- The Pharmacy class
- The main program

```

#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

class PatientAccount {
private:
    int daysInHospital = 0;
    double totalCharges = 0.0;
    static constexpr double DAILY_RATE = 1000.0;

public:
    void setDaysInHospital(int days) {
        if (days < 0) {
            days = 0;
        }
        daysInHospital = days;
    }

    int getDaysInHospital() const {
        return daysInHospital;
    }

    void addCharge(double amount) {
        if (amount < 0) {
            return;
        }
        totalCharges += amount;
    }

    double getTotalCharges() const {
        return totalCharges;
    }

    double getDailyRate() const {

```

```

        return DAILY_RATE;
    }

    double getStayCharges() const {
        return daysInHospital * DAILY_RATE;
    }
};

class Surgery {
private:
    struct Item {
        std::string name;
        double charge;
    };

    std::vector<Item> surgeries{
        {"Appendectomy", 8000.0},
        {"Cataract Surgery", 6000.0},
        {"Bypass Surgery", 25000.0},
        {"Knee Replacement", 18000.0},
        {"Tonsillectomy", 4500.0},
    };

public:
    int count() const {
        return static_cast<int>(surgeries.size());
    }

    const std::string& nameAt(int index) const {
        return
surgeries.at(static_cast<std::size_t>(index)).name;
    }

    double chargeAt(int index) const {
        return
surgeries.at(static_cast<std::size_t>(index)).charge;
    }

    void updateAccount(PatientAccount& account, int index)
const {
        account.addCharge(chargeAt(index));
    }
};

class Pharmacy {
private:
    struct Item {
        std::string name;
        double price;
    };

    std::vector<Item> meds{
        {"Antibiotic", 120.0},
        {"Pain Reliever", 45.0},
        {"Insulin", 80.0},
        {"Blood Pressure Med", 65.0},
        {"Antacid", 25.0},
    };

public:
    int count() const {
        return static_cast<int>(meds.size());
    }

    const std::string& nameAt(int index) const {
        return meds.at(static_cast<std::size_t>(index)).name;
    }
};

```

```

double priceAt(int index) const {
    return meds.at(static_cast<std::size_t>(index)).price;
}

void updateAccount(PatientAccount& account, int index)
const {
    account.addCharge(priceAt(index));
}
};

static int readIntInRange(const std::string& prompt, int
minValue, int maxValue) {
    for (;;) {
        std::cout << prompt;
        int value;
        if (std::cin >> value) {
            if (value >= minValue && value <= maxValue) {
                return value;
            }
        } else {
            std::cin.clear();
        }
        std::cin.ignore(10000, '\n');
        std::cout << "Invalid input. Please try again.\n";
    }
}

int main() {
    PatientAccount account;
    Surgery surgery;
    Pharmacy pharmacy;

    std::cout << "Enter number of days spent in the hospital: ";
    int days;
    while (!(std::cin >> days) || days < 0) {
        std::cin.clear();
        std::cin.ignore(10000, '\n');
        std::cout << "Invalid input. Enter a non-negative
integer: ";
    }
    account.setDaysInHospital(days);

    bool checkedOut = false;
    while (!checkedOut) {
        std::cout << "\nMenu\n";
        std::cout << "1. Enter a type of surgery\n";
        std::cout << "2. Enter a type of medication\n";
        std::cout << "3. Check patient out\n";

        int choice = readIntInRange("Choose an option (1-3):
", 1, 3);
        switch (choice) {
            case 1: {
                std::cout << "\nSurgery Types\n";
                for (int i = 0; i < surgery.count(); ++i) {
                    std::cout << (i + 1) << ". " <<
surgery.nameAt(i) << " ($" << std::fixed
<< std::setprecision(2) <<
surgery.chargeAt(i) << ")\n";
                }
                int s = readIntInRange("Select surgery (1-" +
std::to_string(surgery.count()) + "): ", 1,
surgery.count());
                surgery.updateAccount(account, s - 1);
                break;
            }
        }
    }
}

```

```

        case 2: {
            std::cout << "\nMedication Types\n";
            for (int i = 0; i < pharmacy.count(); ++i) {
                std::cout << (i + 1) << ". " <<
pharmacy.nameAt(i) << " ($" << std::fixed
                << std::setprecision(2) <<
pharmacy.priceAt(i) << ")\n";
            }
            int m = readIntInRange(
                "Select medication (1-" +
std::to_string(pharmacy.count()) + "): ", 1,
                pharmacy.count());
            pharmacy.updateAccount(account, m - 1);
            break;
        }
        case 3:
            checkedOut = true;
            break;
        default:
            break;
    }
}

double total = account.getStayCharges() +
account.getTotalCharges();
std::cout << "\nCheckout\n";
std::cout << "Days in hospital: " <<
account.getDaysInHospital() << "\n";
std::cout << "Daily rate: $" << std::fixed <<
std::setprecision(2) << account.getDailyRate() << "\n";
std::cout << "Stay charges: $" << std::fixed <<
std::setprecision(2) << account.getStayCharges() << "\n";
std::cout << "Other charges (surgery + medication): $" <<
std::fixed << std::setprecision(2) << account.getTotalCharges()
<< "\n";
std::cout << "Total charges: $" << std::fixed <<
std::setprecision(2) << total << "\n";

return 0;
}

```